Documentación y Manual Desarrollo Instrumento Experimental para *performance* Audiovisual

Organismo Central-Vic de Investigación e Innovación
Universidad de Valparaíso

Índice

Acerca de este documento **3**
Que es este documento . . . 3
Como usar este documento . . . 3
Estructura de la información . . . 3

Introducción **4**

Captura Gestual **4**
Introducción . . . 4
Frameworks e Interfaces . . . 5
Frameworks . . . 5
Interfaces de programación . . . 5
MediaPipe . . . 6
Python Implementación . . . 6
Captura gestual de manos . . . 7
Mensajes OSC (Open Sound Control) . . . 11
Tecnologías Alternativas . . . 16
MediaPipe en OpenFrameworks . . . 16
PoseNet . . . 20
P5.js . . . 20
ml5.js . . . 20

Navegación Gestual de Corpus de Datos **20**
Supercollider y FluCoMa . . . 20
Instalación Supercollider y FluCoMa . . . 21
Supercollider IDE . . . 23
Descripción Técnica Supercollider . . . 23
Sintaxis . . . 23
sclang y scsynth . . . 25
Clasificación de elementos sonoros espectralmente en dos dimensiones . . . 26

Clasificación Sonora Espectralmente **26**
Primeros pasos 27
Detección `onset` y Segmentación 28
Análisis de los Segmentos (Descriptores) 30
FluCoMa Dataset 30
Descriptores 30
Tratamiento del *DataSet* 32
Visualización y Navegación del Corpus de Datos 34
Visualización y Navegación con el Ratón 34
Visualización y Navegación Externa 36

Control con Redes Neuronales **40**
Regresión 40
Control de Instrumento Musical con la Pose 41
Detección de Pose Corporal con MediaPipe 41
Regresión usando SuperCollider y FluCoMa 42

Conclusion: Posibles expansiones **49**

Acerca de este documento

Que es este documento

Este texto pretende presentar una documentación acabada y un manual de uso que se enfoca en el uso de herramientas para la integración de inteligencia artificial para la performance audiovisual, clasificación de elementos multimedia, procesamiento de imagen, sonido y control interactivo en tiempo real. El texto busca presentar de forma concisa estas tecnologías mediante la documentación del proceso de creación del instrumento experimental. Por otro lado, debido a la imposibilidad de contener toda la información y conocimiento usado en su desarrollo, el documento es un punto de partida para que el usuario pueda profundizar al presentar de forma ordenada y clara un compendio de tecnologías disponibles y referencias de artículos, libros y material online disponible. Dicho lo anterior y debido al gran tamaño de información que el desarrollo de este instrumento requiere, este es un documento de consulta en el cual el usuario/participante de las sesiones prácticas pueda volver a los tópicos y procesos vistos previamente. En su mayor parte, este documento puede ser usado sin haber participado en las sesiones prácticas. Este documento no es un texto que intenta abarcar todo el conocimiento teórico relacionado con los tópicos necesarios para el desarrollo del instrumento experimental.

Como usar este documento

Este documento debe ser usado como un manual/tutoría y referencia para usar las tecnologías presentadas en los contextos de transferencia de conocimiento. Muchas de las tecnologías usadas en el desarrollo son desarrolladas para ser usadas en diversas plataformas. El documento presentara las diversas alternativas, particularmente porque los diversos ejemplos ayudarán a entender el funcionamiento de estas tecnologías.

Estructura de la información

Este documento presenta diversos tipos de información relevante al desarrollo del proyecto. La información se presentara de la siguiente forma:

- Figuras que contienen información de programación gráfica.
- Bloque de código como el siguiente:

```
//----code---//
line 1
line 2
etc...
```

Listing 1: Código de muestra

- Listas de instrucciones.
- Enlaces hacia los archivos y ejemplos que están contenidos en un repositorio [1] git.

[1] https://github.com/PaoloMolinari/IEPA

Introducción

El desarrollo de un instrumento experimental para performance audiovisual requiere el uso de un avanzado desarrollo tecnológico. Al ser una instancia de introducción de estás tecnologías, su desarrollo se hizo a través de tecnologías de alto nivel ya existentes, además de modelos de inteligencia artificial especializados disponibles con licencias abiertas MIT, GPL-2 o GPL-3 entre otras.

La revisión de estas tecnologías se hizo de forma práctica, lo que ayuda a comprender mecanismos y conceptos teóricos que son la base de estas tecnologías . La primera parte de este documento indaga en los sistemas y frameworks[2] para a captura gestual como MediaPipe, openFrameworks, Python, a través de la implementación de un sistema de captura gestual. En esta parte analizaremos los datos que nos entregan la captura de gestos y revisaremos estrategias de filtrado y simplificación de datos.

En una segunda parte se desarrollara temas de comunicación, y en particular OSC o Open Sound Control, un protocolo para la comunicación de datos y mensajes en tiempo real entre aplicaciones de *software* y *hardware*.

La tercera sección de este documento se concentra en tecnologías de entrenamiento de redes neuronales y los diferentes usos a través de plataformas middleware agnóstico a sistemas y plataformas o frameworks que funcionan dentro de plataformas multimedia como supercollider y pure-data, como lo es FluComa. En estos sistemas se verán dos de las mas comunes aplicaciones en el uso de machine learning y redes neuronales supervisadas y no supervisadas, como lo son la regresión y la clasificación. Si bien estos procesos estará enfocados en audio, son similares a lo que se puede usar en colecciones y control de vídeo, sin embargo, desprovistos de la dificultad extra facilitando el aprendizaje de los flujos de trabajo, los procesos y el lenguaje.

Finalmente, en la cuarta sección se implementará el sistema en su totalidad uniendo las diferentes partes mediante protocolos de comunicación.

Captura Gestual

Introducción

La estimación gestual o de poses se refiere a técnicas de visión por computador que detecta figuras humanas en imágenes y vídeos de tal forma que se puede determinar las diversas articulaciones del cuerpo humano como puede verse en la figura 1 (**??**). De esta forma, se puede identificar por ejemplo donde está el hombro de una persona en una imagen. Estas tecnologías no reconocen quien está en la imagen, estos sistemas no entregan información que sirva para la identificación de personas asociado con la detección de poses. El algoritmo simplemente estima donde las articulaciones del cuerpo humano están en las coordenadas de la imagen.

[2] Un framework es un conjunto de herramientas, estructuras y reglas que proporcionan un marco de trabajo predefinido para el desarrollo de software. En lugar de comenzar desde cero, los programadores pueden aprovechar este marco para acelerar la creación de aplicaciones, asegurando que el código siga principios de diseño robustos y estructurados.

La estimación de pose tienen muchos usos, desde instalaciones interactivas que reaccionan a los movimientos del cuerpo a sistemas de realidad aumentada, y animación entre muchos otros. Existen muchos sistemas alternativas dé detección de pose de código abierto, el desarrollo de tecnologías de inteligencia artificial y machine learning han permitido la liberación del uso de hardware especializado y equipo sofisticado de alto valor y difícil de implementar.

Existen muchas librearías y frameworks que que realizan la estimación de pose, sin embargo nosotros nos concentraremos en dos tecnologías: MediaPipe y PoseNet. Ambas librerías serán usadas de forma nativa usando la inbrterfaz Python que proporcionan, y además ser usadas en frameworks de C++ como openFrameworks para desarrollar aplicaciones locales de alta velocidad, y en p5.js para desarrollar aplicaciones web.

Frameworks e Interfaces

Frameworks

A medida que vamos conociendo las tecnologías que usaremos nos encontraremos frecuentemente con la palabra frameworks en sistemas de programación. Desde esto surge la pregunta acerca de ¿que es un framework? Un frameworks puede ser pensado como un conjunto de métodos o funciones y variables que proporcionan una interfaz de alto nivel para programar. Un framework también puede ser pensado como una infraestructura para que sea mas fácil comenzar a construir y desarrollar nuestros propios programas. En general, los frameworks son tecnologías convenientes que sin embargo nos enseña y obliga a hacer y construir de una forma determinada por el diseño del framework.

En este proyecto nosotros usaremos solo frameworks como OpenFrameworks y P5.js son frameworks para programación multimedia para los lenguajes de programación C++ y Javascript respectivamente. Estos frameworks nos ayudan implementar de forma rápida programas gráficos para el computador y el navegador web, sin tener que implementar nuestros propios sistemas de visualización, o con la cámara, la interacción con el ratón y o el teclado por ejemplo. De esta forma los frameworks nos proporcionan una utilidad.

Interfaces de programación

Si los frameworks nos proporcionan utilidades para programar existen otro tipo de utilidades que pueden ser pensadas incluyo de forma intercambiables con los frameworks, pero que sin embargo tienen la diferencia en que el núcleo de los procesos que estos utilizan fueron programados en otro lenguaje de programación de más bajo nivel. Media|Pipe y PoseNet pueden ser considerados frameworks o mas bien interfaces que usan un sinnúmero de funciones y variable predeterminas que usan el lenguaje de programación Python que fue programado en C++. Esto es similar a como Pytorch y TensorFlow fueron construidos. Sin bien estos últimos proporcionan librerías nativas para ser usadas en programación en C++, lo mas común es usar las interfaces en Python, el cual es un lenguaje de programación de más alto nivel.

MediaPipe

Python Implementación

MediaPipe tiene una implementación Python nativa. Python es un lenguaje de alto nivel que permite fácilmente a un programador primerizo o con experiencia con otros lenguajes comenzar a programar.

Para hacer programas que capturen nuestros gestos con Python se necesitan otras librerías aparte de MediaPipe o PoseNet. A medida que usemos librerías y desarrollemos diferentes programas pueden aparecer incompatibilidades y errores, lo que puede ser extremadamente problemático. Esto se debe a que a medida que instalamos librerías y programas Python, cada uno de estos requieren especificas versiones de otras librerías. Debido a que Python no exija *back-compability*, es decir que la nueva versión de la librería sea compatible con las versiones anteriores, como lo es el lenguaje de programación C++ por ejemplo, para cada desarrollo se necesitan las versiones específicas con las cuales el programa o la librería fue desarrollado. Es por esto que se ocupan *enviroments*, o ambientes de trabajo los cuales se activan y se trabaja independiente de los paquetes instalados en el sistema, o los instalados para el desarrollo de otras aplicaciones. Para crear estos ambientes de trabajo usaremos la herramienta Conda, una herramienta de código abierto para el manejo de paquetes multi-plataforma, que hace mas fácil el acceso para las herramientas y librerías de programación con versiones específicas, independientes de las demandas del sistema operativo u otros desarrollos que eventualmente podrías tener conflictos con las versiones que necesitamos para nuestro proyecto.

Para instalar conda en Fedora Linux, abrimos un terminal y escribimos:

```
sudo dnf install conda
```

para sacar el enunciado `(base)` del prompt del terminal en el mismo terminal escribimos:

```
conda config --set auto_activate_base false
```

Ahora crearemos una carpeta en la cual trabajaremos y creamos nuestro primer ambiente de trabajo:

```
  mkdir mediapipe
  conda create --name mediapipe python= 3.12
```

esto creará un directorio en `/home/usuario/.conda/envs/mediapipe`. El ambiente que creamos especifica la versión de Python que usaremos, que en este caso es la 3.12, la mayor usada por Mediapipe.

Activamos nuestro ambiente de trabajo y una vez que estemos dentro de el instalamos MediaPipe. En este proceso se instalaran un sinnúmero de librerías necesarias para el funcionamiento de MediaPipe.

```
  conda activate mediapipe
  pip install mediapipe
```

una vez que tengamos activado nuestro ambiente de trabajo o enviroment. Para editar el código recomiendo usar el editor de texto zed, que se puede descargar acá. Zed es

un editor de texto relativamente moderno que necesita una mínima configuración para reconocer los enviroments, para manejar los proyectos y por último auto completar el código, sin ser particularmente pesado como los opciones más comunes VScode.

Captura gestual de manos

Toda la documentación de Mediapipe se encuentra en esta página, y las referencias que en ella se hacen al código es el que usaremos en estos ejemplos. Con estas referencias, se podrían implementar todos los las soluciones que tiene Mediapipe para aplicar rápidamente inteligencia artificial y aprendizaje de maquina en nuestras aplicaciones. Algunos de estas soluciones permiten customizar los modelos para tener nuestras propios gestos, sonidos u texto. Dicho esto, el workflow de Mediapipe se podría resumir en cuatro partes:

- Mediapipe Taks: La API que proporciona soluciones multi plataforma para ser utilizadas en nuestros programas.
- Mediapipe Models: Modelos pre-entrenados listos para ser usados.
- MediaPipe Model Maker: Soluciones para crear nuestros propios modelos
- MediaPipe Studio: Una solución para visualizar, evaluar y benchmarking de modelos.

Con motivos de tiempo y espacio, nos concentraremos en la captura de las posiciones de las articulaciones (landmarks) y los gestos de las manos. Los gestos son una serie de posiciones pre-determinadas usando el modelo pre-entrenado. Hay que tener en cuenta que al entrenar nuestros propios modelos la capa del modelo predeterminado que contienen los gestos predeterminados se “reescribe”. Debido a que la documentación presenta el código necesario para crear las implementaciones de forma coherente para todas las soluciones los ejemplos de captura gestual de manos que desarrollaremos nos permitirán fácilmente desarrollar aplicaciones de captura de landmarks faciales y corporales.

La implementación será la de *live video*, para reconocer en tiempo real con el vídeo entrante por una cámara los gestos de nuestras manos. Para esto usaremos la librería openCV[3], la cual no permite capturar y transformar las imágenes de la cámara.

Lo primero que haremos es detectar las coordenadas de las articulaciones. Para esto importaremos las librerías en nuestro programa

```
import cv2 as cv  #importamos openCV as cv
import time  #importamos la libreria time
import mediapipe as mp # importamos mediapipe as mp
from mediapipe.tasks import python #importamos los tasks
from mediapipe.tasks.python import vision # importamos task de vision
```

[3] OpenCV es una biblioteca libre de visión artificial originalmente desarrollada por Intel. OpenCV significa Open Computer Vision (Visión Computacional Abierta). Desde que apareció su primera versión alfa en el mes de enero de 1999, se ha utilizado en una gran cantidad de aplicaciones, y hasta 2020 se la sigue mencionando como la biblioteca más popular de visión artificial. Detección de movimiento, reconocimiento de objetos, reconstrucción 3D a partir de imágenes, son sólo algunos ejemplos de aplicaciones de OpenCV.

Definimos que tasks de visión vamos a utilizar, en este caso reconocimiento de gestos o GestureRecognition

```
BaseOptions = mp.tasks.BaseOptions
GestureRecognizer = mp.tasks.vision.GestureRecognizer
GestureRecognizerOptions = mp.tasks.vision.GestureRecognizerOptions
GestureRecognizerResult = mp.tasks.vision.GestureRecognizerResult
VisionRunningMode = mp.tasks.vision.RunningMode
```

Después creamos una función `callback` que será llamada cada vez que el modelo reconozca un gesto. En este caso nuestra función callback escribía en la consola el objeto de datos que arroja el modelo:

```
  def print_result(result: GestureRecognizerResult, output_image: mp.Image, timestamp_ms: int)
    :
        print('gesture recognition result: {}'.format(result))

```

Ahora definimos el camino a el modelo que usaremos, en este caso el predeterminado, y creamos las siguientes opciones, las cuales serán usadas por la instancia del reconocedor de gestos que declararemos posteriormente. El modelo con los gestos predeterminados puede ser descargados en este enlace.

```
#definimos una variable model_path con un string con la direccion del modelo en nuestro disco
    duro
  model_path = './gesture_recognizer.task'

  # definimos nuestras opciones
  options = GestureRecognizerOptions(
      # camino al modelo  de AI en el disco duro
      base_options=BaseOptions(model_asset_path=model_path),
     # modo live video
     running_mode=VisionRunningMode.LIVE_STREAM,
     # la funcion callback que usaremos
     result_callback=print_result)
```

ahora inicializaremos la cámara de vídeo para alimentar el modelo de reconocimiento de gestos:

```
# inicializamos la captura de ideo
cap = cv.VideoCapture(0)
# si el dispositivo de captura no abre
if not cap.isOpened():
# escribimos el mensaje que el dispositivo no se puede abriri y salimos
    print("Cannot open camera")
    exit()

```

finalmente creamos nuestra instancia llamada `recognizer` usando las opciones

```
    with GestureRecognizer.create_from_options(options) as recognizer:
    #el código de detección lo pondremos aquí

```

después de creada la instancia obtenemos los cuadros del vídeo de entrada con un bucle infinito `while`, y dentro de el obtenemos una imagen cada cuadro desde la captura de vídeo, la cual es convertida a una imagen que reconozca MediaPipe para ser ingresada como parámetro de entrada de la función `recognize_async` cuyos parámetros de entrada son el cuadro y un *timestamp*, que refiere al tiempo desde que comenzó a correr el programa. Este último parámetro no es necesario si el modo es imagen fija.

HP v178b

```python

with GestureRecognizer.create_from_options(options) as recognizer:
    # bucle infinito
    while True:
        # captura cuadro por cuadro
        ret, frame = cap.read()
        # si el cuadro no es recibió correctamente
        # es decir ret no es true
        # enviamos un mensaje y salimos del bucle
        if not ret:
            print("Can't receive frame (stream end?). Exiting ...")
            break

        # si ret es true, es decir recibimos un cuadro
        #convertimos la imagen del cuadro
        # a un tipo de imagen mediaPipe SRGB
        mp_image = mp.Image(image_format=mp.ImageFormat.SRGB, data=frame)

        #ingresamos la imagen como parámetro
        # al método recognize_async
        recognizer.recognize_async(mp_image, time.time_ns() // 1_000_000)

        # mostramos la imagen de la cámara
        cv.imshow('frame', frame)

        # si pulsamos la tecla "q" salimos del bucle
        if cv.waitKey(1) == ord('q'):
            break
# cerramos la captura
# y destruimos todas las ventanas abiertas
cap.release()
cv.destroyAllWindows()
```

Ahora volvamos al modelo y observemos que información de salida nos entrega. Esto es importante si queremos usar esta información y enviarla a través de OSC, debido a que tenemos que extraer la información relevante y estructural el mensaje a ser enviado. Para mayor detalle acerca del modelo se puede consultar el siguiente documento.

Lo primero que podemos ver es que el modelo reconocerá los siguientes gestos:

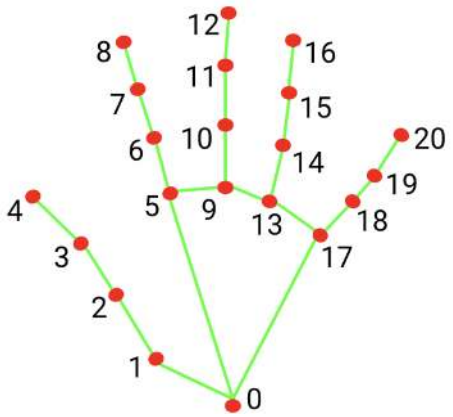
```
0 - Unrecognized gesture, label: Unknown
1 - Closed fist, label: Closed_Fist
2 - Open palm, label: Open_Palm
3 - Pointing up, label: Pointing_Up
4 - Thumbs down, label: Thumb_Down
5 - Thumbs up, label: Thumb_Up
6 - Victory, label: Victory
7 - Love, label: ILoveYou
```

Si estos son los gestos, la pregunta que nace es de que forma nos entrega la información el modelo. En nuestra función callback podremos verla, debido a que enviamos a la consola de Python el objeto de salida del modelo. La forma es la siguiente:

```
GestureRecognizerResult(
  gestures = [[ Category(
    index = 1,
    score = 0.6449271440505981,
    display_name = 'Open_Palm',
    category_name = 'Open_Palm')
    ]],
  handedness=[[ Category(
    index=1,
```

```
    score=0.9102575778961182,
    display_name='Left',
    category_name='Left'
  )]],
  hand_landmarks = [[
    NormalizedLandmark(
      x=0.8542457222938538,
      y=0.7173023223876953,
      z=3.450798828907864e-07,
      visibility=None,
      presence=None,
      name=None),
    NormalizedLandmark(
      x=0.7731325030326843,
      y=0.6579955816268921,
      z=-0.01647113263607025,
      visibility=None,
      presence=None,
      name=None),
    ...
```

Podemos observar que la información esta dividida en tres grupos, `gestures`, `handedness` y `hand_landmarks`. El grupo `gestures` nos entrega el índice de la mano que encontró, si son dos manos esta puede ser `0` o `1` de acuerdo al orden de aparición. El campo `score` es el margen de error estimado del reconocimiento del gesto. Finalmente los campos `display_name` y `category_name` nos entregan el nombre del gesto encontrado. Si no reconoce ninguno entrega un resultado `none`. `handedness` nos entrega si es la mano izquierda o derecha. De igual forma que `gestures` nos entrega el índice correspondiente a la mano, el `score` que es el margen de error estimado del reconocimiento de que mano es, y finalmente `display_name` y `category_name` nos entrega el identificador `Left` para la mano izquierda y `Right` para la mano derecha. Finalmente el grupo `hand_landmarks` nos entrega las coordenadas normalizadas[4] Este grupo nos entrega 21 landmarks normalizados como se muestran en la figura 1.



0. WRIST
1. THUMB_CMC
2. THUMB_MCP
3. THUMB_IP
4. THUMB_TIP
5. INDEX_FINGER_MCP
6. INDEX_FINGER_PIP
7. INDEX_FINGER_DIP
8. INDEX_FINGER_TIP
9. MIDDLE_FINGER_MCP
10. MIDDLE_FINGER_PIP
11. MIDDLE_FINGER_DIP
12. MIDDLE_FINGER_TIP
13. RING_FINGER_MCP
14. RING_FINGER_PIP
15. RING_FINGER_DIP
16. RING_FINGER_TIP
17. PINKY_MCP
18. PINKY_PIP
19. PINKY_DIP
20. PINKY_TIP

Figura 1: Lista de los 21 landmarks de la mano y a que articulación corresponden

[4]Normalizada se refiere cuando tamaños de distintas escalas se transforma a escalas entre 0 y 1. Si en este caso las coordenadas son en el espacio de la imagen en píxeles, es decir el ancho y alto de la imagen, sus coordenadas normalizadas son las coordenadas del punto en píxeles dividido por el ancho y el alto.

Mensajes OSC (Open Sound Control)

Si bien obtenemos los mensajes en nuestro terminal o consola Python, estos datos no los podemos usar directamente en otras aplicaciones. Para esto usamos la librería `Python-osc` que nos permite enviar mensajes OSC fuera de nuestra aplicación. Open Sound Control (OSC) es un protocolo para conectar en red sintetizadores, ordenadores y otros dispositivos multimedia con fines como la interpretación musical o el control de espectáculos. Las ventajas de OSC son la inter-operabilidad, la precisión, la flexibilidad y una mejor organización y documentación. La primera especificación se publicó en marzo de 2002. `Python-OSC` es una biblioteca diseñada para implementar servidores y clientes `OSC` utilizando Python puro. Esta biblioteca facilita la comunicación entre aplicaciones a través de mensajes OSC, que consisten en una dirección y valores asociados. Características principales Protocolos de comunicación Soporte `UDP`: Permite enviar y recibir mensajes `OSC` a través de `UDP`. Soporte `TCP`: Proporciona funcionalidad para la comunicación `OSC` a través de `TCP`. Gestión de mensajes Despachador: Gestiona el enrutamiento de los mensajes `OSC` entrantes a los gestores correspondientes. Creación de mensajes: Simplifica la creación de mensajes `Osc` varios con tipos de datos, incluyendo enteros, floats, cadenas y más.

para instalar la librería simplemente vamos al terminal con el conda enviroment activado y escribimos

```
pip install python-osc
```

una vex instalado importamops la libreria en nuestro programa

```
from pythonosc import udp_client

```

Hay que considerar que estamos importando sólo el cliente UDP debido a varias razones que lo hacen conveniente a pesar de sus limitaciones. El protocolo de datagramas de usuario (en inglés: User Datagram Protocol o UDP) es un protocolo que permite el envío de data de forma rápida en redes IP sin establecer previamente una conexión, dado que el propia data incorpora suficiente información en su cabecera. Hay que considerar, que el protocolo no tiene confirmación ni control de flujo, por lo que los paquetes pueden adelantarse unos a otros. Además, tampoco se sabe si hubo entrega o recepción debido a que no existe una confirmación si los paquetes han llegado correctamente.

Ahora una vez importado el cliente UDP desde la librería python-osc tenemos que configurar el cliente de OSC asignando la dirección IP y el puerto en que el programa cliente, o el dispositivo de red recibirá los mensajes que estamos creando. Esto se hace de la siguiente forma:

```
# OSC Configuration
OSC_IP = "127.0.0.1"
OSC_PORT = 9000
osc_client = udp_client.SimpleUDPClient(OSC_IP, OSC_PORT)
```

Ahora tememos que crear o dar forma a los mensajes OSC con la información que nos entrega Mediapipe. Un mensaje OSC puede ser un numero de punto flotante o `float`, o un arreglo o lista de valores de diferente tipo, lo que incluye una cadena de caracteres junto con valores de punto flotante. Cada mensaje OSC tiene como encabezado una dirección que tiene la siguiente forma `/encabezado1/`, el cual puede tomar la forma de

direcciones tal como navegamos por las carpetas del sistema operativo en el terminal `/enbamezado1/encabezado2/encabezado3/`. De esta forma, podemos ordenar los mensajes jerárquicamente de acuerdo a la estructura de la información que queramos enviar. Si consideramos la estructura que nos entrega MediaPipe podríamos pensar en la siguiente estructura de mensajes OSC:

```
/gestures/index/
/gestures/score/
/gestures/display_name/
/gestures/category_name/



/handedness/index/
/handedness/score/
/handedness/display_name/
/handedness/category_name/



/hand_landmarks/NormalizedLandmark/0/x
/hand_landmarks/NormalizedLandmark/0/y
/hand_landmarks/NormalizedLandmark/0/z
/hand_landmarks/NormalizedLandmark/0/visibility/
/hand_landmarks/NormalizedLandmark/0/presence/
/hand_landmarks/NormalizedLandmark/0/name/
.
.
.
/hand_landmarks/NormalizedLandmark/20/x
/hand_landmarks/NormalizedLandmark/20/y
/hand_landmarks/NormalizedLandmark/20/z
/hand_landmarks/NormalizedLandmark/20/visibility/
/hand_landmarks/NormalizedLandmark/20/presence/
/hand_landmarks/NormalizedLandmark/20/name/
```

Sin embargo no toda la información entregada por Mediapipe es relevante. Por otro lado quizás queramos estructural la información en el programa o computador cliente. Por lo tanto, la otra opción es enviar una gran lista con la información relevante de la siguiente forma:

```
/hand/
  index,
  gestures category_name,
  handedness category_name,
  NormalizedLandmark 0 x,
  NormalizedLandmark 0 y,
  NormalizedLandmark 0 z,
  ...
  NormalizedLandmark 20 x,
  NormalizedLandmark 20 y,
  NormalizedLandmark 20 z
```

Toda la estructuración de la información entregada por Mediapipe y la formación de los mensajes OSC la haremos en nuestra función *callback* llamada `results`, la cual es llamada al momento que Mediapipe reconoce los landsmarks de las manos. El uso de esta función por el reconocedor de gestos de Mediapipe fue definida en las opciones

que le entregamos. El código sería el siguiente:

```
def result(result: GestureRecognizerResult, output_image: mp.Image, timestamp_ms: int):
    #declarar variables globales de la funcion
    # para el lado de la mano, el id de la mano y el gesto
    hand_side = ""
    handID = ""
    gesture = ""

    # si el subcampo de los rsultados del reconocimeinto de gestos
    # es "handedness"
    if result and result.handedness:
        # para cada mano enumere los campos de handedness
        for hand_idx, handedness in enumerate(result.handedness):
            # para el handedness de cada mano
            for hand in handedness:
                # guardamos en nuestras variables
                # el index de la mano y el lado de esta (derecha o izquierda)
                handID = hand.index
                hand_side = hand.category_name
    # para el gesto de cada mano si pertenece al grupo "gestures"
    if result and result.gestures:
        # enumeramos las componentes de la lista "gestures"
        for hand_idx, gesturesResult in enumerate(result.gestures):
            # para cada mano copiamos en nuestra variable gesto
            # el gesto de cada mano
            for gest in gesturesResult:
                # copiamos la categoria de gesto en nuestra variable
                # gesture
                gesture = gest.category_name

    # ai el subitem es hand_landmarks
    if result and result.hand_landmarks:
        # creamos una lista llamada landmarks-data
        landmarks_data = []
        # creamos una lista que contendra todos
        # los datos que enviaremos por el mensaje OSC
        all_data = []
        # para cada grupo de landmarks de cada mano
        for hand_idx, landmarkResult in enumerate(result.hand_landmarks):
            # para cada landmark de cada grupo de landmarks
            for i, land in enumerate(landmarkResult):
                # populamos el array o lista con el indice y las coordenadas
                # x, y, z de cada landmark.
                landmarks_data.extend([i, land.x, land.y, land.z])
            # populamos el array "all_data" con el id de la mano
            # el lado de la mano, el gesto, y el array con las
            # coordendas de los landmarks
            all_data.extend([handID, hand_side, gesture])
            all_data = all_data + landmarks_data
            # enviamos el mensaje OSC
            osc_client.send_message(f"/hand/{hand_idx}", all_data)


```

el resultado es un mensaje que tienen los siguientes campos:

`[0, 'Right', 'Open_Palm', [0, 0.200, 0.675, 0.363, 1, 0.271, 0.618, -0.007, 2, 0.314, 0.544, -0.003 ... 20, 0.315, 0.291, 0.005]]`

El código final es el siguiente:

```
# importar openCV como cv
import cv2 as cv
# importar librería de herramientas temporales
import time

# importar el cliente udp
# desde la librería pythonosc
from pythonosc import udp_client

# importar Mediapipe como mp
```

```python
import mediapipe as mp

# importar python y vision
# de la librería MediaPipe.tasks
from mediapipe.tasks import python
from mediapipe.tasks.python import vision

# definir los task que usaremos
# en este caso GestureRecognizer
BaseOptions = mp.tasks.BaseOptions
GestureRecognizer = mp.tasks.vision.GestureRecognizer
GestureRecognizerOptions = mp.tasks.vision.GestureRecognizerOptions
GestureRecognizerResult = mp.tasks.vision.GestureRecognizerResult

# definir el modo en que haremos correr tasks
VisionRunningMode = mp.tasks.vision.RunningMode

# configuración OSC
# definición de IP, en este caso localhost
# definición del puerto
OSC_IP = "127.0.0.1"
OSC_PORT = 9000

# definición de instancia del cliente OSC
# con el respectivo IP y puerto
osc_client = udp_client.SimpleUDPClient(OSC_IP, OSC_PORT)

# función callbak
def result(result: GestureRecognizerResult, output_image: mp.Image, timestamp_ms: int):
    # declaración variables globales de la función
    # para el lado de la mano, el id de la mano y el gesto
    hand_side = ""
    handID = ""
    gesture = ""

    # si el subcampo de los resultados del reconocimiento de gestos
    # es "handedness"
    if result and result.handedness:
        # para cada mano enumere los campos de handedness
        for hand_idx, handedness in enumerate(result.handedness):
            # para el handedness de cada mano
            for hand in handedness:
                # guardamos en nuestras variables
                # el index de la mano y el lado de esta (derecha o izquierda)
                handID = hand.index
                hand_side = hand.category_name
    # para el gesto de cada mano si pertenece al grupo "gestures"
    if result and result.gestures:
        # enumeramos las componentes de la lista "gestures"
        for hand_idx, gesturesResult in enumerate(result.gestures):
            # para cada mano copiamos en nuestra variable gesto
            # el gesto de cada mano
            for gest in gesturesResult:
                # copiamos la categoría de gesto
                # en nuestra variable gesture
                gesture = gest.category_name

    # si el subitem de los resultados es "hand_landmarks"
    if result and result.hand_landmarks:
        # creamos una lista llamada landmarks_data
        landmarks_data = []
        # creamos una lista que contendrá todos
        # los datos que enviaremos por el mensaje OSC
        all_data = []
        # para cada grupo de landmarks de cada mano
        for hand_idx, landmarkResult in enumerate(result.hand_landmarks):
            # para cada landmark de cada grupo de landmarks
            for i, land in enumerate(landmarkResult):
                # populamos el array o lista con el índice y las coordenadas
                # x, y, z de cada landmark.
                landmarks_data.extend([i, land.x, land.y, land.z])
            # populamos el array "all_data" con el id de la mano
            # el lado de la mano, el gesto, y el array con las
```

```
            # coordenadas de los landmarks
            all_data.extend([handID, hand_side, gesture])
            all_data = all_data + landmarks_data
            # enviamos el mensaje OSC
            osc_client.send_message(f"/hand/{hand_idx}", all_data)

# declara variable que guarda el camino al modelo
model_path = '/home/pemb/python/mediapipe/gesture_recognizer.task'

# opciones para el reconocedor de gestos
options = GestureRecognizerOptions(
    # lugar del modelo
    base_options=BaseOptions(model_asset_path=model_path),
    # el modo en que se hará correr el
    # reconocedor de gestos
    # este puede ser, STILL_IMAGE, VIDEO y LIVE_STREAM
    running_mode=VisionRunningMode.LIVE_STREAM,
    # cantidad de manos a capturar
    num_hands=2,
    # definición de la función callback
    result_callback=result)

# Inicialización captura de vídeo con openCV
cap = cv.VideoCapture(0)
# Si el dispositivo de vídeo no abre
if not cap.isOpened():
    # mensaje y salida
    print("Cannot open camera")
    exit()

# Imprimir que se esta inicializando
print("Starting hand tracking with LIVE_STREAM mode...")
# Escribir en la consola el ip y puerto
# en el cual esta siendo enviado el mensaje OSC
print(f"Sending OSC messages to {OSC_IP}:{OSC_PORT}")

# Comenzar la instancia del reconocedor de gestos
with GestureRecognizer.create_from_options(options) as recognizer:
    # bucle infinito
    while True:
        # Captura cuadro por cuadro
        ret, frame = cap.read()
        # si el cuadro no es recibió correctamente
        # es decir ret no es true
        # enviamos un mensaje y salimos del bucle
        if not ret:
            print("Can't receive frame (stream end?). Exiting ...")
            break

        # Si ret es true, es decir recibimos un cuadro
        # convertimos la imagen del cuadro
        # a un tipo de imagen mediaPipe SRGB
        mp_image = mp.Image(image_format = mp.ImageFormat.SRGB, data = frame)

        # Ingresamos la imagen como parámetro
        # al método recognize_async
        recognizer.recognize_async(mp_image, time.time_ns() // 1_000_000)

        # mostramos la imagen de la cámara
        cv.imshow('frame', frame)

        # si pulsamos la tecla "q" salimos del bucle
        if cv.waitKey(1) == ord('q'):
            break
# Cerramos la captura
# y destrumimos todas las ventanas abiertas
cap.release()
cv.destroyAllWindows()
```

Tecnologías Alternativas

MediaPipe en OpenFrameworks

La Interfase de MediaPipe es sólo en Python. Sin embargo, esta puede ser usada en C++ usando la librería PyBind, que proporciona una transparente interoperabilidad entre Python y C++. Pybind es una librería de C++ liviana que contienen solo archivos header, es decir .h, los cuales exponen tipos de C++ en Python y vice versa.

MediaPipe en openFrameworks ya está implementado como un `addon`. Un `addon` es una librería externa que se incluye en un programa de openFrameworks. OF esta diseñado de tal forma que la inclusión de la librería sea de forma mas o menos inmediata, sin tener que lidiar con mecanismos de compilación y enlace propios de C++. La implementación de MediaPipe en OF también incluye la transición de datos a través de OSC, que nos permite usar los datos obtenidos de la captura de gestos en cualquier otro programa.

Si bien los pasos para hacer funcionar MediaPipe en C++ parecieran menos intuitivos. Lo primero que debemos hacer es bajar la versión la última versión "nightly" de OF en el siguiente enlace[5], en nuestro directorio `home`. Después descomprimimos el archivo,

```
tar -zxvf of_v[fecha_en_que_fue_compilado]_linux64_gcc6_release.tar.gz
```

el comando anterior creará una carpeta en el directorio `home`. Ingresamos a esta carpeta,

```
cd ~/of_v[fecha_en_que_fue_compilado]_linux64_gcc6_release/
```

Dentro encontraremos la siguiente estructura de directorios:

```
  ls -l
  # resultados
  addons
  apps
  docs
  examples
  libs
  other
  scripts
  projectGenerator
```

La función y contenido de cada carpeta es la siguiente:

`addons` - carpeta en que se disponen los addons o las librerías externas de OF; `apps` - lugar donde creamos nuestras aplicaciones; `docs` - documentación de OF; `examples` - carpeta con los ejemplos de OF; `libs` - librerías externas necesarias para compilar OF; `other`; `scripts` - scripts para facilitar la compilación de OF; `projectGenerator` - programa para generar provectos de OF.

Ahora vamos a compilar OF. Compilar es el proceso de convertir código de programación en código máquina o algún otra forma de bajo nivel, que será ejecutado por el computador. Esto abre una interrogante: ¿porque tenemos que compilar OF si nuestros programas que hagamos usándolos igualmente vamos a tener que compilarlos? Este proceso no se hace solo para producir programas en código máquina, sino que

[5] https://github.com/openframeworks/openFrameworks/releases/tag/nightly

además librerías que puedan ser usados por otros programas. De esta forma, no tenemos que compilar OF cada vez que hacemos un cambio en nuestro programa.

Compilar es una gran diferencia entre lenguajes de programación como C++ y Python o Javascript, debido a que estos últimos son lenguajes cuyo código es interpretado por un interprete del lenguaje. En el caso de Python es el programa interprete Python que puede ser llamado en el terminar, y en el caso de Javascript es el navegador de internet. Si bien esto es conveniente por que se puede ir probando rápidamente y desarrollar esquemas y prototipos de forma rápida, estos son considerablemente mas lentos que un programa compilado en código máquina. Es por esto, que para aplicaciones en las cuales el tiempo de ejecución es determinante como juegos o programas multimedia, el uso de lenguajes como C++ es predominante.

Para compilar OF ingresamos a la carpeta `scripts` sucesivamente de acuerdo al sistema en donde lo vamos a instalar. En nuestro caso, usando Fedora Linux, sería lo siguiente:

```
cd scripts/linux/fedora/
```

Una vez que estamos dentro de la carpeta ejecutamos los dos scripts que existen dentro de la carpeta, los cuales van a instalar automáticamente las librerías necesarias de sistema para compilar OF y los codecs multimedia necesarios para reproducir sonido y vídeo en nuestros programas.

```
  ./install_dependencies.sh
  ./install_codecs.sh
```

Básicamente estos scripts ejecutan en comando `dnf` de nuestro sistema. Una vez que las librerías estén instaladas, ejecutamos el script de compilación que se encuentra en la carpeta `linux`, un nivel atrás de la carpeta `fedora`:

```
  cd ../
  ./compileOF.sh
```

Si queremos podemos compilar también el generador de proyectos, sin embargo esto lo haremos de forma manual para entender mejor como funciona OF. Para compilar el generador de proyectos:

```
  ./compilePG.sh
```

Si el proceso de compilación no arroja errores, una vez terminado, podríamos compilar uno de los ejemplos sin problemas. Vamos a la carpeta de ejemplos de vídeo, debido a que la captura de vídeo es importante para nuestro proyecto. Sin embargo podríamos usar cualquier ejemplo dentro de la carpeta `examples`:

```
cd examples/video/videoGrabberExample/
```

para compilar usamos `make`:

```
make
```

Una vez compilado, se generaría un archivo ejecutable con el nombre del proyecto (el nombre de la carpeta que lo contiene, en este caso `videoGrabberExample`, en la carpeta `bin`. Podríamos hacer correr el programa desde esta carpeta, pero es mas conveniente usar `make run`, desde la carpeta base del proyecto. De esta forma para ejecutar el programa ejecutamos el siguiente comando en nuestro terminal:

```
make run
```

Ahora deberíamos vernos desde la cámara del computador. Esto quiere decir que podemos capturar vídeo. Si todo funciona bien ahora clonaremos el repositorio del addon `ofxMediaPipePython` en nuestra carpeta addons:

```
cd openFrameworks/addons/
git clone https://github.com/design-io/ofxMediaPipePython.git
```

Una vez copiado el repositorio instalaremos `MediaPipe` usando el script que vienen en el `addon`

```
cd openFrameworks/addons/ofxMediaPipePython
./InstallMediaPipe.sh
```

Copiaremos el ejemplo con `OSC sender` que se encuentra dentro del `addon` en la capeta `apps/myApps`:

```
cp -R openFrameworks/addons/ofxMediaPipePython/MediaPipeOscSenderExample openFrameworks/apps/
    myApps/MediaPipeOscSenderExample
cd openFrameworks/apps/myApps/MediaPipeOscSenderExample
```

y modificaremos el archivo `config.make` que se encuentra dentro de la carpeta del proyecto

```
ls #enter
addons.make  addons.make~  bin  config.make  Makefile  obj  src
```

```
nano config.make
```

Descomentamos la línea `PROJECT_DEFINES =`, y agregamos `OF_ADDON_HAS_OFX_OSC`

```
################################################################################
# PROJECT DEFINES
#   Create a space-delimited list of DEFINES. The list will be converted into
#   CFLAGS with the "-D" flag later in the makefile.
#
#               (default) PROJECT_DEFINES = (blank)
#
#   Note: Leave a leading space when adding list items with the += operator
################################################################################
PROJECT_DEFINES = OF_ADDON_HAS_OFX_OSC
```

Una vez hecho esto deberíamos poder compilar el ejemplo:

```
cd openFrameworks/apps/myApps/MediaPipeOscSenderExample
make
make run # para hacer correr el programa
```

Deberíamos poder ver el programa con la imagen

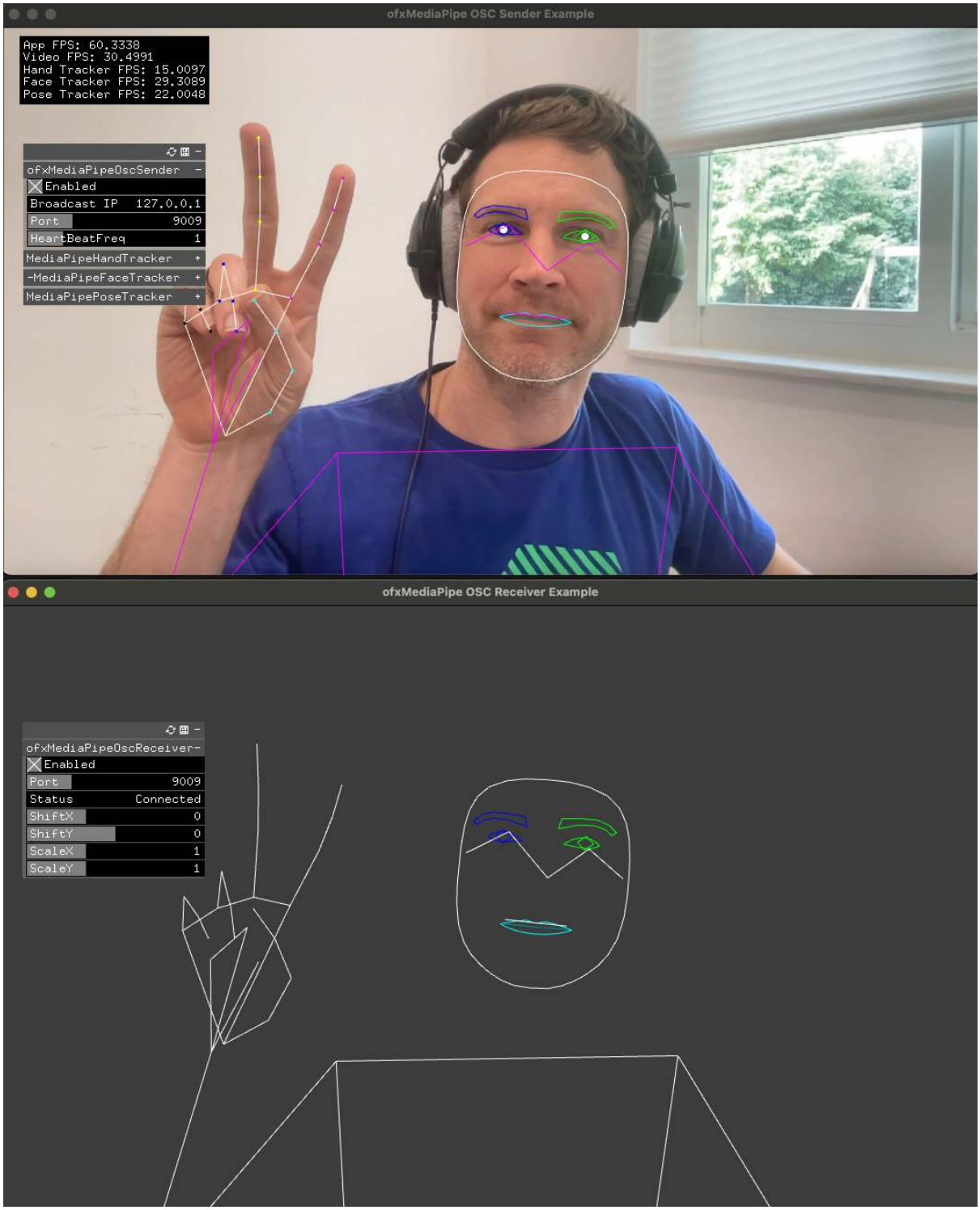


Figura 2: Imagen del ejemplo del programa de ofxMediaPipePython corriendo

A diferencia con la aplicación que creamos usando Python los autores de `ofxMediaPipePython` crearon una aplicación que captura los landmarks de la cara, el cuerpo y las manos. En la izquierda de la ventana se puede desactivar los diferentes elementos de inferencia.

Para crear nuestro propio programa en OF usando MediaPipe, se puede copiar la carpeta con el proyecto vació que nos proporciona OF como un nuevo proyecto con otro nombre.

PoseNet

Existe además otras opciones de captura gestual y de pose que usan otras tecnologías que también pueden ser usados. Se podría decir que una de las opciones mas fáciles de usar e implementar es la que usa poseNet, en p5.js y su librería ml5.js.

P5.js

`p5.js` es una biblioteca de `JavaScript` de código abierto diseñada para hacer la programación creativa accesible y fácil de aprender. Inspirada en `Processing`, un entorno de programación visual creado en 2001 pensado para artistas y diseñadores, `p5.js` lleva esa misma filosofía al navegador web usando `JavaScript`. Esta tecnología se usa principalmente para crear gráficos y animaciones interactivas, arte generativo, visualizaciones de datos, experiencias visuales y sonoras en el navegador. Lo que hace especial a p5.js es que tiene una sintaxis sencilla, ideal para principiantes debido a que funciona directamente en el navegador, sin necesidad de instalar nada, haciéndolo muy popular en entornos educativos y artísticos.

ml5.js

Por otro lado, `ml5.js` es una biblioteca de código abierto con el objetivo de hacer que el aprendizaje automático sea accesible para una amplia audiencia de artistas, programadores creativos y estudiantes. Inspirándose en Processing y `p5.js`, `ml5.js` busca tender un puente entre la complejidad técnica del aprendizaje automático y la creatividad de los principiantes y artistas. La biblioteca proporciona acceso a algoritmos y modelos de aprendizaje automático en el navegador, construyendo sobre `TensorFlow.js`. `ml5.js` se inspiró directamente en `p5.js`. De hecho, ambas bibliotecas funcionan muy bien juntas: `p5.js` se encarga de la parte visual e interactiva, mientras que `ml5.js` aporta las capacidades de aprendizaje automático. En resumen, `p5.js` es una herramienta pensada para que artistas, diseñadores y personas sin mucha experiencia en programación puedan crear cosas visuales e interactivas en la web de forma sencilla.

En particular para obtener los `landmarks` de las manos la documentación se encuentra en este enlace.[6]

Navegación Gestual de Corpus de Datos

Supercollider y FluCoMa

En la tutoría usaremos Supercollider y Flucoma.

SuperCollider es un entorno de programación de código abierto diseñado específicamente para la síntesis de audio en tiempo real y la composición algorítmica. Es muy popular en el mundo de la música electrónica, el arte sonoro y la investigación académica.

[6] https://docs.ml5js.org/#/reference/handpose

Supercollider tiene dos partes principales que trabajan juntas:

- sclang – el lenguaje de programación con el que escribes el código
- scsynth – el servidor de audio que genera el sonido en tiempo real

Supercollider es usado para la síntesis de sonido y música generativa, composición algorítmica, performances en vivo (live coding), instalaciones sonoras e interactivas, e investigación en música por computadora.

SuperCollider está estrechamente relacionado con FluCoMa. FluCoMa es un conjunto de herramientas diseñado para usarse dentro de SuperCollider (y también en Max y Pure Data), permitiendo aplicar aprendizaje automático y análisis de sonido dentro de ese entorno.

¿Qué es FluCoMa?, FluCoMa significa *Fluid Corpus Manipulation* y es un conjunto de herramientas diseñadas para la manipulación creativa de grandes colecciones de sonido usando técnicas de aprendizaje automático y análisis de audio. FluCoMa fue desarrollado principalmente en el Centre for Digital Music (C4DM) de la Universidad de Londres y financiado por el European Research Council, con el objetivo de permitir a músicos y artistas sonoros a explorar y navegar grandes colecciones de sonido (corpus), analizar características del sonido automáticamente, separar y segmentar audio de forma inteligente, y aplicar aprendizaje automático (*machine learning*).

Sus herramientas principales son la segmentación automática de audio, análisis de sonido mediante descriptores de audio, separación de fuentes sonoras, herramientas de *Machine Learning* para clasificar y organizar sonidos, y finalmente herramienta de visualización de datos.

A diferencia de otras herramientas de *machine learning*, FluCoMa está pensado para la exploración creativa, no solo el análisis técnico de corpus de datos y sonido, sin que sea necesario ser experto en programación ni en ML, fomentando el descubrimiento accidental e inspiración.

Instalación Supercollider y FluCoMa

La instalación de supercollider en Fedora Linux y en Linux en general se hace a través de un proceso que se llama compilar, es decir, al igual que openFrameworks, es pasar el programa a código maquina.

Las instrucciones se encuentran en este enlace, sin embargo a continuación se enumeraran los pasos.

Instalar los paquetes de sistema necesarios para compilar Supercollider:

```
sudo dnf install gcc jack-audio-connection-kit-devel libsndfile-devel cmake fftw-devel libXt-
    devel git gcc-c++ libX11-devel qt5-qtwebkit-devel qt5-qtlocation-devel qt5-qtsensors-devel
     systemd-devel libatomic avahi-devel qt5-qttools-devel emacs alsa-lib-devel
```

Obtenemos el código fuente de Supercollider

```
git clone https://github.com/supercollider/supercollider.git
```

nos movemos al directorio de Supercollider

```
cd supercollider
```

y descargamos los submódulos

```
git submodule init && git submodule update
```

una vez descargados los componentes, creamos un directorio llamado `build` en donde se alojaran los archivos resultantes del proceso de compilación

```
mkdir build
cd build
```

hacemos correr el `CMake` para producir el archivo de compilación

```
cmake -DCMAKE_BUILD_TYPE=Release ..
```

y compilamos

```
make
```

Una vez terminada la compilación, procedemos a instalar supercollider como super usuario

```
sudo make install
sudo ldconfig
```

probamos si esta todo funcionando haciendo correr el programa desde el Desktop Environment o ejecutando el siguiente comando

```
sccide
```

Si todo funciona bien, procedemos a instalar los plugins de supercollider, que nos entregan mas objetos que podemos usar en nuestros patchs. en una carpeta fuera de la carpeta supercollider clonamos el repositorio de sc3-plugins

```
git clone https://github.com/supercollider/sc3-plugins.git
```

después nos movemos a la carpeta del repositorio y ejecutamos los siguientes comandos

```
git submodule init && git submodule update
mkdir build && cd build
cmake -DSC_PATH=/lugar/codigo/SuperCollider/ -DCMAKE_INSTALL_PREFIX=/usr/local -
    DCMAKE_BUILD_TYPE=Release ..
make
sudo make install
sudo ldconfig
```

abrimos supercollider y ejecutamos la siguiente línea

```
{VOSIM.ar(Impulse.ar(100), 500, 3, 0.99)}.play
```

Si se emite sonido, los plugins fueron instalados correctamente.

La instalación de FluComa es más fácil debido a que existen paquetes pre-compilados que necesitan ser copiados a una carpeta específica usada por supercollider para agregar extensiones. Para esto ejecutamos las siguientes líneas en el terminal:

```
cd ~/.local/share/SuperCollider/Extensions/
wget https://github.com/flucoma/flucoma-sc/releases/download/1.0.9/FluCoMa-SC-Linux-x64.tar.gz
tar -zxvf FluCoMa-SC-Linux-x64.tar.gz -C FluidCorpusManipulation
```

Para comprobar si esta funcionando abrimos la IDE de Supercollider y ejecutamos los siguiente

```
FluidFilesPath();
```

Deberíamos ver la carpeta en la consola de supercollider.

Supercollider IDE

Supercollider viene con su propio IDE (Integrated developed enviroment). La parte izquierda del IDE es donde escribiremos el código. En la parte superior derecha se encuentra la documentación, en la cual podremos acceder a la explicación de cada una de las clases de supercollider que usaremos, además de ejemplos. En la parte inferior derecha podremos ver la consola de Supercollider, en la cual podremos decirle al programa que nos muestre información así como también es en este lugar donde se nos dirá si en nuestro programa existe un error.

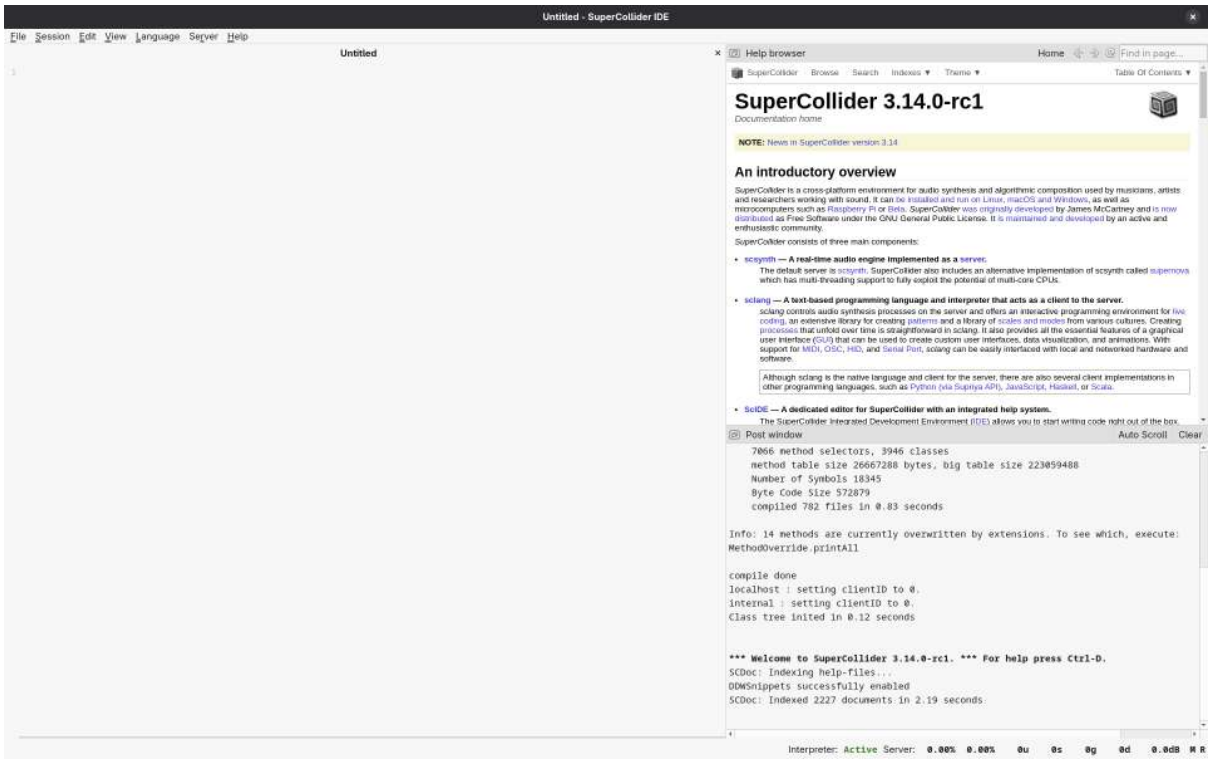


Figura 3: Supercollider IDE

Descripción Técnica Supercollider

Sintaxis

La sintaxis de SuperCollider tiene una lógica muy consistente. En las siguientes líneas enumeraremos los elementos principales.

Los comentarios se hacen de la siguiente forma:

```
// Esto es un comentario de una linea

/* Esto es un comentario
   de multiples lineas */
```

Las variables se declaran de la siguiente forma:

```
// Variables locales (del nombre debe comenzar con letra minuscula)
var x = 10;
var nombre = "hola";

// Variables globales (empiezan con ~)
~miVariable = 42;
```

```
~buffer = Buffer.read(s, "sonido.wav");
```

Las variables pueden contener instancias de clases, así como también tipos de datos:

```
var numero    = 42;          // Entero
var decimal   = 3.14;        // Float
var texto     = "hola";      // String
var simbolo   = \hola;       // Symbol
var arreglo   = [1, 2, 3];   // Array
var booleano  = true;        // Boolean
var buf = Buffer.read(s, "sonido.wav"); // Instancia de una clase
```

Los bloques de código en SuperCollider se encierran en paréntesis `()`. El código que se encuentra dentro de los paréntesis se ejecuta como una unidad línea por línea. Si las variables son globales, estas al ser declaradas a nivel de environment o ambiente, serán reconocidas en otros bloques. Si las variables declaradas son locales, estás serán consideradas válidas solo dentro del bloque.

```
(
~gvar = 1; // variable global
var x = 5; // variable local
var y = 10;
(x + y).postln; // imprime el resultado en la consola
)
```

Los mensajes y métodos en SuperCollider usan la misma sintaxis de otros lenguajes de programación orientada a objetos, donde los métodos se llaman con un punto,

```
10.squared;       // 100
"hola".toUpper;   // "HOLA"
[1,2,3].reverse;  // [3,2,1]
[1,2,3].size;     // 3
```

Las funciones se definen y se llaman de la siguiente forma,

```
// Definir una funcion
var miFuncion = { |x, y|
    x + y
};

// Llamar la funcion
miFuncion.value(3, 4); // retorna 7
```

Por último, considerando sólo lo que haremos en la tutoría, debemos notar como hacer un instrumento de sonido básico. Por ejemplo, dos síntesis de sonido básicos que usan distintos UGens (Unit Generators) para generar sonido puede ser las siguientes,

```
// Reproducir un tono simple
(
{
    var sig = SinOsc.ar(440, 0, 0.3); // onda seno a 440Hz
    sig
}.play;
)

// Ruido con filtro
(
{
    var ruido = WhiteNoise.ar(0.2);
    var filtrado = LPF.ar(ruido, 1000); // filtro pasa bajos
    filtrado
}.play;
)
```

Otros conceptos básicos que debemos tener en cuenta son los siguientes:

- Los bloques se ejecutan con Ctrl+Enter (o Cmd+Enter en Mac)
- El servidor de audio debe iniciarse primero con `s.boot`
- `.postln` es tu mejor amigo para depurar (imprime valores en la consola)
- Los `UGens` terminan en `.ar` (audio rate) o `.kr` (control rate)

sclang y scsynth

En SuperCollider existe una característica que es importante entender antes de comenzar a desarrollar los sistemas con FluCoMa. En SuperCollider existe el sclang y el scserver.

sclang (SuperCollider Language) es el lenguaje de programación con el que el usuario interactúa. Es donde se escribe y ejecuta el código, el que interpreta y procesa las instrucciones, el que se comunica con scsynth enviándole mensajes vía OSC (Open Sound Control). En resumen es el espacio donde se maneja la lógica, los patrones, las secuencias y el control del tiempo. Mientras que el scsynth (SuperCollider Synthesizer), es el servidor de audio donde se produce el sonido. No se interactúa directamente con scsynth, sino mas bien este recibe instrucciones de sclang y las convierte en audio. De esta forma maneja todos los UGens (osciladores, filtros, efectos, etc.), y opera en tiempo real con muy baja latencia. El scsynth corre como un proceso que está separado del lenguaje, el cual es extremadamente eficiente para el procesamiento de señal.

Dicho esto, existen estructuras de datos que funcionan a nivel de sclang y otras a nivel de scsynth. Como consecuencia, existen métodos que nos permiten traspasar información entre sclang y scsynth. Por ejemplo los `Buffers` pertenecer a `scsynth`, mientras que un `Array` pertenece al lenguaje `sclang`. Ambos pueden considerarse como contenedores de datos secuenciales. Así, para trasladar la información de un `Array` a un `Buffer` se utiliza `loadCollection`,

```
(
var miArray = [0.1, 0.5, -0.3, 0.8, -0.1, 0.4];

~miBuffer = Buffer.loadCollection(s, miArray, 1, {
    "Buffer cargado!".postln;
    ~miBuffer.postln; // muestra info del buffer
});
)
```

o usando `setn`, lo que es más rápido pero necesitamos que el `Buffer` haya sido declarado anteriormente y reservar el espacio para este en la memoria,

```
(
~miBuffer = Buffer.alloc(s, 6, 1, { // reservar espacio
    ~miBuffer.setn(0, [0.1, 0.5, -0.3, 0.8, -0.1, 0.4]); // llenar con datos
    "Buffer llenado!".postln;
});
)
```

Por otro lado, para pasar la información de un `Buffer` a n `Array` se utilizan los métodos `getToFloatArray` o `loadToFloatArray`,

```
(
~miBuffer.loadToFloatArray(action: { |array|
    array.postln; // imprime el array en la consola
    ~miArray = array; // guardarlo en una variable global
```

```
});
)
```

También se puede utilizar el método `getn`, si queremos acceder a solo una parte del `Buffer`

```
(
// leer 6 valores empezando desde el indice 0
~miBuffer.getn(0, 6, { |valores|
    valores.postln;
});
)
```

Así existen muchas otras formas de comunicación entre el lenguaje y el servidor que usaremos para traspasar información entre estos. La intensión con el código anterior, es mostrar esta diferencia y como los datos operan en diferentes espacios .

Clasificación de elementos sonoros espectralmente en dos dimensiones

Clasificación Sonora Espectralmente

Esta parte de la tutoría intenta guiar en la construcción de un programa de exploración de un corpus de sonidos que puede ser un archivo de sonido o una serie de sonidos de una variada cantidad de fuentes. Como referencia se pueden ver software comerciales como CataRT, LjudMAP o XO.

La idea es poder explorar una colección de sonidos relativamente desconocida en un espacio visual. Aprovechar el posicionamiento espacial de las muestras para una exploración creativa o expresiva que informe una compresión del corpus de sonido que responda a lógicas intuitivas.

La tutoría comenzará mostrando cómo segmentar un archivo de sonido y analizar cada segmento describiendo su intensidad y estructura espectral. Esta información se utiliza luego para controlar un mecanismo de reproducción guiada por los descriptores, los cuales son procesados y convertidos en coordenadas, de modo que cada segmento de sonido pueda ser ubicado en un espacio 2D. Esto crea el espacio visual que puede navegar con el ratón o con nuestros gestos capturados usando MediaPipe. Cada componente fundamental del programa estará separado para que se pueda experimentar y explorar el flujo de trabajo de una manera personalizada.

Como forma de resumen el diagrama de flujo del algoritmo sera el siguiente:

Fuente de sonido o archivo de audio de varios sonidos –>
Segmentación (`FluidBufOnsetSlice`...) –>
Análisis de descriptores (`Loudness, SpectralShape, MFCC`...) –>
Reducción de dimensionalidad (`UMAP` / `PCA`) –>
Visualización (`FluidPlotter`) –>
Navegación e interacción creativa (Ratón, `MediaPipe`)

Primeros pasos

Lo primero que se hará es definir un bloque de código en el que iniciaremos el servidor de sonido con opciones particulares. La primera es aumentar el numero de Buses y la segueta la cantidad de memoria que el servidor usara. En relación a los Buses, hay que considerar a SuperCollider y su ethos de herramienta musical y sonora, en donde muchos de su paradigma viene de metáforas de instrumentos y herramientas de musica electrónica. La clase `Bus` es una de ellas. Un bus en audio se refiere a la ruta de señal que permite se alimentado con múltiples señales de audio al mismo tiempo que permite obtener esas señales desde otras partes del programa. En SuperCollider existen Buses de audio y también Buses de control que nos permiten controlar simultáneamente varias pistas de audio individuales, como ajustes de volumen, ecualización y efectos basados en el tiempo como la reverberación.

Una vez iniciado el servidor ejecutamos `s.meter` y `s.plotTree` para ver el nivel de las salidas de audio y el árbol que muestra los procesos que están funcionando en el servidor de audio respectivamente.

```
(
// Incrementamos el numero de Buses
s.options.numBuffers = 1024 * 16;
// Incrementamos la cantidad de memoria del servidor
s.options.memSize = 8192 * 64;
//Iniciamos el servidor
s.boot;

s.meter;
s.plotTree;
)
```

Ahora definiremos el directorio que aloja nuestras muestras de sonido. Declaramos una variable global que contiene la ruta al directorio llamada `folder`. En este caso, usaremos el directorio de sonidos que viene con la distribución de FluCoMa, que es llamado por el método `FluidFilesPath()`. Después cargamos en la memoria cada archivo de audio que se encuentra en la ruta, y los asignamos a un Buffer, cargando todos los archivos en el buffer usando el método `FluidLoadFolder`. Este método incluye la información sobre los puntos de inicio y fin de cada sample, el número de canales y las tasas de muestreo, los cuales son almacenados luego en el índice como un `IdentityDictionary`. Un `IdentityDictionary` es una estructura de datos en SuperCollider que almacena información como pares clave-valor, similar a un diccionario o tabla de búsqueda. Es una de las estructuras de datos más utilizadas en SuperCollider.

```
(
// Definimos la ruta del directorio en donde están los archivos de sonido
~folder = FluidFilesPath();

// FluidLoadFolder carga en un Buffer cada archivo de audio en la ruta de principio a fin, y
    le asocia la informacion sobre los puntos de inicio y fin, el numero de canales y las
    frecuencias de muestreo.

~loader = FluidLoadFolder(~folder).play(
  s,
  {"done loading folder".postln;}
);
)
```

Como último paso de la primera parte convertiremos los archivos contenidos en la variable `loader` a un canal si estos son estéreos usando la clase `FluidBufCompose`,

una utilidad para manipular el contenido de `Buffers`.

```
(
// si el Buffer ~loader tiene mas de un canal
if(~loader.buffer.numChannels > 1){
  // declaramos un Buffer de salida
  ~src = Buffer(s);
  //Para cada miembro del Buffer ~loader haga (do) para cada canal:
  ~loader.buffer.numChannels.do{
    // argumento para contener el numero de canales
    arg chan_i;

    // "FluidBufCompose" es una utilidad para manipular el contenido de buffers.
    // Proporcionamos el rango de muestras y canales para ser copiados,
    // asi como tambien, el Buffer destinatario, la copia de canales y la ganacia,

    FluidBufCompose.processBlocking(
      s,
      ~loader.buffer,
      startChan: chan_i,
      numChans: 1,
      gain: ~loader.buffer.numChannels.reciprocal,
      destination: ~src,
      destGain: 1,
      action:{"copied channel: %".format(chan_i).postln}
    );
  };
}{
  "loader buffer is already mono".postln;
  ~src = ~loader.buffer;
};
)
```

Podemos usar el método `FluidWaveform` para ver el contenido del `Buffer`

```
FluidWaveform(~src);
```

Detección `onset` y Segmentación

La detección de *onset* es una tarea crucial en el procesamiento de señales de audio que implica identificar el comienzo de un sonido o un evento musical. Es un componente fundamental en diversas aplicaciones de recuperación de información musical (MIR), como la transcripción de música, el seguimiento del compás y la segmentación del audio. En esta sección, exploraremos la definición y la importancia de la detección de *onset*, los tipos de *onsets* y los desafíos asociados con ella.

Los *onsets* se pueden clasificar en dos categorías generales: percusivos y no-percusivos. Los *onsets* percussivos: Los *onsets* percusivos se caracterizan por un aumento repentino de energía, típicamente asociado a instrumentos como la batería y la percusión. Estos *onsets* suelen estar marcados por un ataque fuerte y un rápido descenso. Los *onsets* no-percusivos se asocian a instrumentos como las cuerdas y los vientos. Estos *onsets* suelen caracterizarse por un aumento más gradual de energía y pueden implicar cambios espectrales complejos.

En FluComa existen varios algoritmos para hacer detecciones de onset que sirven para diferentes fuentes sonoras, nosotros usaremos la clase `FluidBufOnsetSlice`. Esta clase implementa una selección de detectores de eventos sonoros con los cuales se segmenta un archivo de audio. La segmentación se hace en base a la diferencia espectral entre los cuadros de un sample. Voy a hacer un paréntesis para definir lo que es un cuadro en sonido digital. Cada sample del sonido independiente de la cantidad de canales del archivo de audio se le llama un cuadro. La métrica para calcular

la diferencia espectral puede ser elegida desde una selección ya curada, de tal manera que se puede ejecutar la segmentación de material sonoro con amplia gama de características diferentes. El argumento con el que se define la métrica puede ser pasado como número entero o símbolo: `power`, `hfc`, `flux`, `mkl`, `is`, `cosine`, `phase`, `wphase`, `complex`, o `rcomplex`. En nuestro ejemplo usamos el número entero 9 que corresponde a `rcomplex`.

```
(
// Creamos un Buffer para contener los índices
// en los cuales se harán los cortes

~indices = Buffer(s);

FluidBufOnsetSlice.processBlocking(
  s, // El servidor de audio
  ~src, // El Buffer de entrada
  metric: 9, // La métrica del análisis
  threshold: 0.05, // El limite para detectar un evento. 0.0 - 1.0
  indices: ~indices, // El Buffer donde se guardaran los indices en que se realizan los cortes
  action: { // la acción a hacer cuando se encuentra un evento
  "found % slice points".format(~indices.numFrames).postln;
  "average duration per slice: %".format(~src.duration / (~indices.numFrames + 1)).postln;
});
)
```

Podemos visualizar la forma de onda con el lugar de los cortes con la clase `FluidWaveform`.

```
FluidWaveform(~src,~indices);
```

Ahora definiremos un instrumento para reproducir los segmentos al que llamaremos `play_slice`

```
(
~play_slice = {
   // El argumento del instrumento es el índice del corte
   arg index;
      {
      // Creamos dos variables que contiene los índices de entrada y salida usando el Buffer de índices.
         var startsamp = Index.kr(~indices, index);
         var stopsamp = Index.kr(~indices, index + 1);

         // Creamos una variable usando un Phasor para crear una rampa numérica desde el cuadro de entrada del segmento hasta el numero final del segmento en samples con una tasa de reproducción del Buffer ~src
   var phs = Phasor.ar(
    0,
    BufRateScale.ir(~src),
    startsamp,
    stopsamp);

         // Creamos una variable usando BufRd para reproducir el segmento del sample ~src usando como controlador la variable que contiene el Phasor
   var sig = BufRd.ar(1,~src, phs);
         // Declaramos una variable que guarde la duración del segmento en muestras
         var dursecs = (stopsamp - startsamp) / BufSampleRate.ir(~src);
         // declaramos una variable para nuestra envolvente
         var env;
         // Convertimos los valores de dursecs desde muestras a segundos
   dursecs = min(dursecs, 1);
         // Declaramos la envolvente, con un ataque de 0.03 segundos, un sostenimiento de la duración del segmento menos 0.6 segundos y un release de 0.03 seg. Una vez terminado, el instrumento es eliminado del servidor usando el parámetro doneAction: 2
   env = EnvGen.kr(
       Env([0, 1, 1, 0], [0.03, dursecs - 0.06, 0.03]),
     doneAction:2
    );
         // Copiamos la señal para crear un array de audio de dos valores para que la reproducción sea estéreo
         sig.dup * env;
```

Identificador	Data				
Guitarra	0.38	0.86	0.23	0.89	0.12
Sintetizador	0.12	0.34	0.71	0.29	0.88
Trombone	0.54	0.75	0.40	0.45	0.12
Saxofone	0.21	0.42	0.63	0.86	0.98
Ruido	0.51	0.28	0.91	0.47	0.42

```
      }.play;
};
)
```

Ahora podemos reproducir el segmento que queramos pasando el argumento del índice que queremos reproducir de la siguiente forma

```
~play_slice.value(1200);
```

Análisis de los Segmentos (Descriptores)

FluCoMa Dataset

Antes que comencemos con el análisis de los segmentos me gustaría definir que es un DataSet en FluCoMa. Un `FluidDataSet` es un contenedor que asocia data. Cada vez que haya necesidad de crear una colección de datos (como para la entrada de un proceso de aprendizaje automático), o de hacer una provisión para alguna salida, el DataSet es la herramienta. Un DataSet comienza vacío y se pueden agregar cualquier número de puntos a él. Un punto está compuesto por dos piezas de información: un identificador y un número de datos asociado al identificador, que puede ser uno hasta cientos o mas. Los datos son cualquier número de bits de información numérica almacenados en un buffer. El identificador es una cadena o símbolo asociado a los datos. En resumen, el objeto DataSet almacena datos de manera asociativa.

Hay ciertas consideraciones que debemos tener en cuenta. Primero, que el tamaño de la data usada para cada punto necesita ser uniforme, y es el primer punto que se agrega al objeto DataSet es el que dictará cuantos números serán asociados para cada punto. Segundo, los identificadores deben ser únicos. Estos no se pueden repetir. Tercero, cuando la data es transformada y pasada entre instancias de objetos de FluCoMa, los identificadores se mantienen, lo que significa que se pueden trackear hacia atrás al comienzo del pipeline de los procesos.

A continuación podemos ver un pequeño ejemplo de DataSet con 5 puntos. Cada identificador es un nombre de instrumento, y podríamos imaginar que los datos asociados a cada uno de estos identificadores podrían ser valores de descriptor o parámetros que describe el timbre de un sonido.

Descriptores

Para el análisis usaremos la clase `FluidBufMFCC`, que es una herramienta de FluCoMa que analiza un `buffer` de audio y extrae los Coeficientes Cepstrales en las Frecuencias de Mel (MFCC por sus siglas en inglés: *Mel-Frequency Cepstral Coefficients*). Es

una de las herramientas de análisis más utilizadas en el procesamiento de audio y el aprendizaje automático. Los MFCCs son una representación matemática del timbre de un sonido, independientemente de su altura o volumen. Los descriptores obtenidos son útiles para clasificar tipos de sonido (voz, instrumentos, ruido), identificar timbres similares dentro de un corpus, agrupar sonidos con características parecidas y alimentar algoritmos de *machine learning*.

Para implementar el análisis de descriptores hacemos el siguiente bloque de código,

```
(
// Creamos una variable que contenga un "FluidDataSet"
~analyses = FluidDataSet(s);

// Pasamos los elementos del Buffer ~indices como un Array de números de punto flotante y procedemos a hacer la siguiente acción para cada uno de sus miembros definiendo una función.

// NOTA: Es importante notar que en Supercollider los Buffer funcionan a nivel de servidor de sonido, mientras que los Array funcionan a nivel de lenguaje. De esta forma, necesitamos usar el método loadToFloatArray.

~indices.loadToFloatArray( action: {
  // Dentro de la función
  // creamos un argumento llamado fa,
        // que contendrá el Array de punto flotante obtenido.

  arg fa;

  // Después creamos tres Buffers. Unos llamado mfccs para guardar los coeficientes de nuestro análisis. Uno llamado stats para guardar las estadísticas de nuestro análisis. Finalmente uno llamado flat, para guardar el resultado del proceso de aplanar las estadísticas a un Buffer unidimensional.

  var mfccs = Buffer(s);
  var stats = Buffer(s);
  var flat = Buffer(s);

  // Para cada elemento del Array (o Buffer). el método "doAdjcentpairs" llama a una función para cada adyacente par de elementos de una colección. El método nos entrega los dos elementos adyacentes de la colección "fa", los cuales son pasados a través de los argumentos "start" y "end", los que representan el corte de cada segmento o onset encontrado. Debido a esto, un segmento tiene un largo en muestras o samples que corresponde al próximo punto de corte menos el actual punto de corte. Finalmente, "doAdjacentPairs" nos entrega el índice.

  fa.doAdjacentPairs{
    arg start, end, i;

    // Obtenemos el largo de cada segmento
    var num = end - start;

    // Usamos la clase "FluidBufMFCC". El algoritmo comprime el espectro de un sonido en un pequeño numero de coeficientes que, cuando son manipulados juntos, describen un contorno general del espectro.

    // NOTA: Las  clases Fluid procesan el Buffer de entrada en el servidor de sonido. En estos casos es usado processBlocking  en vez de process debido a que ejecutara un trabajo que toma una gran cantidad de tiempo, por lo que es necesario capturar el servidor hasta cuando el proceso se termine. En cambio process usa otro hilo thread del procesador para que los otros procesos continúen. El problema aparece cuando código posterior a la salida del proceso mientras este todavía no termina de ejecutarlo, lo que arrojaría un error.

    FluidBufMFCC.processBlocking(
      s,
      ~src,
      start,
      num,
      features: mfccs, //Buffer de salida
      numCoeffs: 13,
      startCoeff:1
    );
```

```
43
44
45      // "FluidBufStats" sumariza estadísticamente una serie de momentos ( o cualquier tipo de
        valor) que es en un Buffer, devolviendo 7 estadísticas para cada canal: el promedio del
        Buffer por canal, la desviación estándar, el skewness o asimetría, la kurtosis o la forma
        en que los datos están distribuidos, además de los valores bajos, los medios y altos. Todo
         estas dimensiones son guardadas en el Buffer "stats"
46
47      FluidBufStats.processBlocking(
48        s,
49        mfccs,
50        stats: stats,
51        select: [\mean]
52      );
53
54      // "FluidBufFlatten" aplana un Buffer multicanal en una dimensión o canal. Esto se usa
        para reestructurar el Buffer de tal forma que puede ser agregado a un "FluidDataSet"
55
56      FluidBufFlatten.processBlocking(
57        s,
58        stats,
59        destination: flat //el Buffer de salida
60      );
61
62      // Finalmente, para cada elemento de flat usando el índice i, agregamos un punto al
        FluidDataSet ~analysis
63      ~analyses.addPoint(i, flat);
64
65  // posteamos en la consola el nombre del identificador que construimos usando el índice i
66      "analyzing slice % / %".format(i + 1,fa.size - 1).postln;
67
68      // Utilizamos este código para que el servidor se sincronice con el cliente del lenguaje
        cada 100 iteraciones
69      //  if((i%100) == 99){s.sync;}
70      //if(i % 100 == 1,{s.sync});
71    };
72
73  // Sincronizamos con el servidor
74    s.sync;
75
76  // Imprimimos en la consola el contenido del FluidDataSet ~analysis
77    ~analyses.print;
78  });
79  )
```

Tratamiento del *DataSet*

Una vez que hemos obtenido los datos, podemos imprimir su contenido en la consola usando `print` los resultados serían los siguientes:

```
1  ~analyses.print;
```

```
1  -> FluidDataSet(3)
2  DataSet 3:
3  points:1473      cols:13
4  0      19.178     39.764     2.9135        ...    -4.4953      1.902    -3.4719
5  1      40.394     27.711    -3.6654        ...    -3.0757    0.89689    -4.8538
6  2     -20.341     20.832     5.1027        ...     3.9201      12.24      -1.51
7         ...
8  1470      61.546     29.493     21.201        ...   -0.84012   -0.44147     1.9325
9  1471      42.922     33.591     19.055        ...      1.484     2.8775     4.5877
10 1472      74.087     12.223    -8.1321        ...     12.029      15.95     3.9632
```

Si observamos FluidDataSet tiene 1473 puntos que coincide con la cantidad de segmentos que obtuvimos desde el *onset detection*. Por otro lado, podemos ver que el *dataset* tiene 13 dimensiones, es decir 13 valores que acompañan a cada punto del *dataset*. Debido a que queremos representar el *dataset* en un espacio bidimensional para poder ser navegado, necesitamos reducir la dimensionalidad del *dataset* a

2 elementos. Para esto usaremos una clase llamada `FluidUMAP`. Esta herramienta de FluCoMa, implementa el algoritmo UMAP (*Uniform Manifold Approximation and Projection*). Su función principal es la reducción de dimensionalidad, es decir, tomar datos con muchas dimensiones y reducirlos a 2 o 3 dimensiones manteniendo las relaciones entre ellos. En nuestro caso, desde el analizas con MFCCs obtenemos 13 coeficientes por cada segmento. Lo que dificulta o hace imposible de visualizar o navegar directamente. UMAP resuelve esto reduciendo esas 13 dimensiones a solo 2, manteniendo los resultados similares del análisis cerca entre sí. La implementación del UMAP sería la siguiente,

```
(
// Creamos una variable ~umapped que contiene un nuevo FluidDataSet
~umapped = FluidDataSet(s);
// Creamos un FluidUMAP con una salida de 2 dimensiones
FluidUMAP(
  s,
  numDimensions: 2, // Dimensiones de salida. valores tipicos son 2 y 3
  numNeighbours: 15, // Vecinos locales a considerar
  minDist: 0.9 // Distancia minima entre puntos en el resultado
).fitTransform(~analyses, ~umapped, action:{"umap done".postln});
)
```

Si imprimimos en la consola el FluidDataSet en la consola obtenemos el siguiente resultado:

```
~umapped.print;
```

```
-> FluidDataSet(4423)
DataSet 4423:
points:1473     cols:2
0     8.0689    14.717
1     6.3515    8.1969
2     13.412    11.418
        ...
1470     2.4777    12.589
1471     4.0533    12.693
1472      4.334    8.6818
```

Si nos fijamos detenidamente en estos valores, podemos observar que su valor es variado, es decir no tienen una escala definida. Para resolver esta dificultad vamos a normalizar el *dataset*. El proceso de normalización refiere a hacer que todos los valores del *dataset* tengan un rango de valores entre 0 y 1. Esto es importante porque podemos manipular mas fácilmente los valores manteniendo la relación entre ellos. Para hacer esto existe una clase llamada *FluidNormalize*

```
(
~normed = FluidDataSet(s);
FluidNormalize(s).fitTransform(~umapped, ~normed);
)
```

Imprimiendo el contenido del `FluisDataSet  normed` en la consola,

```
~normed.print;
```

```
-> FluidDataSet(4425)
DataSet 4425:
points:1473     cols:2
0    0.58133    0.8209
1    0.49971   0.50132
2    0.83524    0.6592
        ...
1470     0.3156   0.71663
1471    0.39048   0.72172
1472    0.40382    0.5251
```

Una vez que obtenemos el *dataset* de dos dimensiones normalizado que mantiene las relaciones obtenidas del análisis de descriptores para cada segmente, procederemos a visualizar nuestro análisis.

Visualización y Navegación del Corpus de Datos

Visualización y Navegación con el Ratón

Para la navegación del corpus de datos usaremos una clase llamada `FluidKDTree`. Esta clase se basa en un algoritmo llamado *KDTree* que tiene como objetivo hacer una búsqueda instantánea de elementos similares. En otras palabras, esta clase organiza la colección de manera inteligente para que cuando se le pregunte ”¿cuál es el elementos más parecido a este punto?”, responda de inmediato sin tener que revisar toda la colección, lo cual es esencial para aplicaciones en tiempo real.

Mas específicamente, `KDTree` (*K-Dimensional Tree*) es una estructura de datos en forma de árbol que organiza puntos en un espacio multidimensional. En FluCoMa se usa principalmente para hacer búsquedas del sonido más cercano a un punto dado de manera muy eficiente.

Su funcionamiento es de la siguiente forma. Imaginemos que tenemos 1000 sonidos distribuidos en un espacio 2D. Si quisieras encontrar el sonido más cercano a un punto dado, podrías compararlo con los 1000 sonidos uno por uno, lo cual es muy ineficiente. KDTree organiza esos sonidos en una estructura de árbol que permite encontrar el más cercano mucho más rápido, sin tener que comparar todos.

Para crear un KDTree, hacemos lo siguiente,

```
// Creamos un KDtree con el dataset normalizado
~tree = FluidKDTree(s).fit(~normed);
```

Una vez que tenemos el `KDTree` de nuestro `FluidDataSet` procedemos a visualizar el *dataset* // usando el método `dump` en el `FluidDataSet` normalizado. El método `dump` almacena los datos del `FluidDataSet` como un diccionario en un objeto `Dictionary`. Tiene como argumento una función que se ejecuta cuando el servidor ha completado el traslado de los datos al diccionario.

```
(
~normed.dump({

  // Declaramos el argumento dict que contendrá
        // el diccionario que almacenara los datos
  arg dict;

  // Creamos una variable point que contendrá un Buffer
        // de dos canales y una variable previous inicializada con nil.

  var point = Buffer.alloc(s, 2);
  var previous = nil;

  // Finalmente visualizamos el diccionario en la consola.

  dict.postln;

.defer es un método en SuperCollider que retrasa la ejecución de una función para que se
    ejecute en el hilo principal de la interfaz gráfica (AppClock), en lugar del hilo actual
    donde se esta ejecutando el código.

  // Creamos una función con en método .defer para crear la componente gráfica.
        // El método .defer retrasa la ejecución de una función
```

```
22        // para que se ejecute en el hilo principal de la interfaz gráfica,
23        // en lugar del hilo actual en donde se esta ejecutando el código.
24
25  defer{
26    // "FluidPlotter", es una clase de FluCoMa para visualizador de datos.
27                // Esta clase tiene un argumento "mouseMoveAction" con el cual podemos
28                // navegar por los datos usando el ratón.
29
30    FluidPlotter(
31      dict: dict,
32      mouseMoveAction: {
33
34        // Declaración argumento view,
35        // x e y para las coordenadas del ratón
36
37        arg view, x, y;
38
39        [x,y].postln;
40
41        // Agregamos las coordenadas x e y
42        // del ratón al Buffer point
43        // de dos canales canales
44        // en el índice 0
45
46        point.setn(0, [x, y]);
47
48        // Usamos el método "kNearest" de
49        // la instancia ~tree de la clase KPTree.
50        // la que nos entrega el punto mas
51        // cercano dado un punto x, y.
52
53        ~tree.kNearest(
54          point, // Punto al que le buscaremos el elemento mas cercano
55          1, // El numero de vecinos que buscara
56          {  // La accion a realizar (una función)
57
58            // La función recibe como argumento
59            // el elemento mas cercanos como argumento
60
61            arg nearest;
62
63            // Si el elemento "nearest" es distinto
64            // al elemento "nearest" anterior encontrado?
65            if(nearest != previous){
66
67              nearest.postln;
68
69              // Highlight el elemento mas cercano
70              view.highlight_(nearest);
71
72              // Llamamos a la función "~play_slice" para
73              // reproducir el segmento de audio
74              // mas cercano
75              ~play_slice.(nearest.asInteger);
76
77              // Copiamos el índice del elemento
78              // mas cercano, a la variable "previous"
79              previous = nearest;
80            }
81          });
82      });
83  }
84 });
85 )
```

Hasta este momento hemos controlado la clasificación de sonidos con el ratón, Sin embargo, si queremos controlar la clasificación externamente, es decir, definir el punto en donde navegaremos por el corpus de datos externamente, necesitamos cambiar el código debido a que la clase `FluidPlotter` solo proporciona una función para el ratón, la cual se ejecuta sólo cuando interactuamos con el ratón.

Visualización y Navegación Externa

Para controlar externamente el punto crearemos un bloque de código para navegar por el `KDTree` a través de mensajes `OSC`. Lo diferente en este código es que el punto de navegación lo dibujaremos sobre la visualización de `FluidPlotter` usando la clase `UserView` que viene incluida en SuperCollider.

```
(
// Creamos variables para la función circle,
// que dibuja un circulo para coordenadas y radio dado.
// Creamos otra variable para una ventana,
// para un objeto UserView, para el Buffer point,
// para el punto previo y para el FluidPlotter.

var w, uv, fp;

var ypos = 0;
var xpos = 0;

// Reservamos el espacio de un Buffer de dos elementos
// lo asignamos a la variable point
var point = Buffer.alloc(s, 2);

// Inicializamos la variable previous con un valor nil o nulo.
var previous = nil;

// Definimos la función que dibujará un círculo
// con tres argumentos, la coordenadas x e y
// y el radio del círculo

var circle = {
  |x, y, r|
  Pen.fillColor = Color.new255(255, 0, 0, 80);
  Pen.strokeColor = Color.red;
  Pen.addArc(x@y + ((r@r)), r, 0, 2pi);
  Pen.fillStroke;
};

// Creamos la ventana y la asignamos a la variable w
// de nombre "Monitor" y tamaño de 800 x 800 píxeles,

w = Window("Monitor", bounds: Rect(0, 0, 800, 800)).front;

// Definimos el color del fondo con total transparencia
w.view.background_(Color.hsv(1, 1, 1, alpha: 0));

// Con el FluidDataSet como diccionario, dibujamos los elementos
// con "FluidPlotter" sin la función ".action". Es decir sin la función
// que activa el ratón.

// Si no damos cuenta el primer parámetro "FluidPlotter"
// es la ventana emparentada.

~normed.dump({
  arg dict;
  ~normed_dict = dict;
  dict.postln;
  defer{
    fp = FluidPlotter(
      w,
      dict: dict,
      xmin: 0,
      xmax: 1,
      ymin: 0,
      ymax: 1,
      bounds: Rect(0, 0, w.bounds.width, w.bounds.height));

    // Inicializamos la clase UserView y la asignamos a la variable uv
    // En la función "drawFunc" dibujamos el circulo llamado a la
    // función circle. Como los valores del FluisDataSet están
    // normalizados, los multiplicamos por el ancho y el alto de la ventana.
```

```
    // El radio de los circulos es 10.

    uv = UserView(w, Rect(0, 0, w.bounds.width, w.bounds.height))
    .drawFunc = { |uvview|
      circle.value(
        xpos * w.bounds.width,
        ypos * w.bounds.height,
        10
      );
    };
  }
});

// Creamos una función OSCdef para recibir mensajes OSC llamada \xy

OSCdef(\xy, {
  arg msg;
        msg.postln;

        // Ingresamos los miembros del Buffer point
        // el segundo y tercer elemento del Array
        // que compone el mensaje OSC.
        // El primer elemento es siempre la ruta del mensaje
  point.setn(0, [msg[1], msg[2]]);

        // Ingresamos el punto a nuestro KDTree
  ~tree.kNearest(point, 1, {
    arg nearest;
    if(nearest != previous){
            // Ingresamos el punto mas cercano encontrado de vuelta
                        // al diccionario ~normed para que nos entregue las coordenadas
                        // del punto encontrado. Esto se debe porque queremos que nuestro
                        // circulo caiga (snapping) sobre el punto mas cercano. De otra forma
                        // el elemento seria reproducido sin que el circulo este sobre el
      # xpos, ypos = ~normed_dict["data"][nearest.asString];
      nearest.postln;
      ~play_slice.(nearest.asInteger);
                        // Usamos el método defer para definir una función que
                        // refresque el UserView o haga un update para que muestre los cambios
      defer{ uv.refresh };
      previous = nearest;
    };
  };
  );

}, '/3/xy');
)

// Finalmente, como modo de prueba si no tenemos un controlador OSC externo
// creamos una función que envíe mensajes OSC a la ruta /3/xy con valores al
// azar entre 0.0 y 1.0

(
m = NetAddr.new("localhost", 57120);
~rndmsg = Routine{loop {
  m.sendMsg("/3/xy", 1.0.rand, 1.0.rand);
  1.wait;
}};
// Hacemos comenzar la rutina
~rndmsg.play;
)
// Detenemos la rutina
~rndmsg.stop;
```

Ahora usar MediaPipe como controlador no es difícil si usamos los gestos como entrada. La idea es asociar un gesto a la suma o resta de una cierta cantidad a cada elemento de la coordenada de navegación. En nuestro ejemplo, la mano derecha controlará el eje `X` y la mano izquierda el eje `Y`. En cada mano, el gesto asociado con el `key` Pointing_Up le sumará 0.01 a la coordenada y Victory, le restará 0.01 a las coordenadas en su respectiva mano. A su vez, Closed_Fist le sumara `0.001` a las coordenadas y

Open_Palm le restará `0.001` en su mano respectiva. De esta forma el código final para controlar la navegación por el corpus de datos usando los gestos capturados en el programa de MediaPipe que creamos tendríamos que reemplazar la función `OSCdef`, por cuatro funciones que reciban los mensajes de las rutas `handedness` y `gesture` que definimos en el programa de Python para cada índice de mano.

```

// Definimos la función OSCdef \gesture0 para la mano con índice 0

OSCdef(\gesture0, {
  arg msg;

        // Preguntamos si la mano con índice 0 es la derecha, y si esto es así
        // controlamos la coordenada x con los gestos. Si no es la mano derecha, controlamos
        // la coordenada y debido a que es la mano izquierda

  if(handedness0 == 'Right'){
    if(msg[1] == 'Closed_Fist', {
      txpos = txpos + 0.001;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Open_Palm', {
      txpos = txpos - 0.001;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Pointing_Up', {
      txpos = txpos + 0.01;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Victory', {
      txpos = txpos - 0.01;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Thumb_up', {
      txpos = rrand(1.0);
      ("X temporal Value: " + txpos).postln});
  }{
    if(msg[1] == 'Closed_Fist', {
      typos = typos + 0.001;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Open_Palm', {
      typos = typos - 0.001;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Pointing_Up', {
      typos = typos + 0.01;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Victory', {
      typos = typos - 0.01;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Thumb_up', {
      typos = rrand(1.0);
      ("Y temporal Value: " + typos).postln});
  };

        // Ingresamos la coordenada al KDTree
  point.setn(0, [txpos, typos]);
  ~tree.kNearest(point, 1, {
    arg nearest;
    if(nearest != previous){
      //nearest.asString.postln;
      # xpos, ypos = ~normed_dict["data"][nearest.asString];
      nearest.postln;
      ~play_slice.(nearest.asInteger);
      defer{ uv.refresh };
      previous = nearest;
    };
  };
  );
}, '/gesture/0');


OSCdef(\gesture1, {
  arg msg;

        // Hacemos lo mismo para la mano con índice 1
```

```
  if(handedness1 == 'Right'){
    if(msg[1] == 'Closed_Fist', {
      txpos = txpos + 0.001;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Open_Palm', {
      txpos = txpos - 0.001;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Pointing_Up', {
      txpos = txpos + 0.01;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Victory', {
      txpos = txpos - 0.01;
      ("X temporal Value: " + txpos).postln});
    if(msg[1] == 'Thumb_up', {
      txpos = rrand(1.0);
      ("X temporal Value: " + txpos).postln});
  }{
    if(msg[1] == 'Closed_Fist', {
      typos = typos + 0.001;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Open_Palm', {
      typos = typos - 0.001;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Pointing_Up', {
      typos = typos + 0.01;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Victory', {
      typos = typos - 0.01;
      ("Y temporal Value: " + typos).postln});
    if(msg[1] == 'Thumb_up', {
      typos = rrand(1.0);
      ("Y temporal Value: " + typos).postln});
    };

  point.setn(0, [txpos, typos]);
  ~tree.kNearest(point, 1, {
    arg nearest;
    if(nearest != previous){
      //nearest.asString.postln;
      # xpos, ypos = ~normed_dict["data"][nearest.asString];
      nearest.postln;
      ~play_slice.(nearest.asInteger);
      defer{ uv.refresh };
      previous = nearest;
    };
  };
  );
}, '/gesture/1');


// Definimos dos funciones OSCdef para cada índice del gesto encontrado
// y guardamos su identificador (si es la mano derecha o izquierda)
// en su respectiva variable handedness0 o handedness1

OSCdef(\handedness0, {
  arg msg;
  handedness0 = msg[1];
}, '/handedness/0');

OSCdef(\handedness1, {
  arg msg;
  handedness1 = msg[1];
}, '/handedness/1');
```

De esta forma logramos analizar un cuerpo de datos sonoro, analizarlo de tal forma que obtenemos descriptores que nos permiten visualizar sus relaciones, y finalmente poder navegar este cuerpo de datos gestualmente.

Hasta acá hemos visto como utilizar gestos para el control de variables de forma di-

recta. Sin embrago que pasaría si usáramos los landmarks, los cuales tienen una gran cantidad de datos; o que sucedería si lo que queremos controlar no tienen dos dimensiones sino que 13 como nuestro MFCC, o 100. Para esto usamos redes neuronales que pueden ser entrenadas supervisadamente para realizar el procesos de regresión, es decir, una red neuronal usada para cambiar la dimensionalidad de datos respecto a otros. A continuación se discutirá un código para entender como podemos implementar estas redes neuronales de acuerdo a las necesidades. Se espera que después de esta explicación, los usuarias puedan modificar el código fácilmente para lograr lo que desea.

Control con Redes Neuronales

Regresión

La regresión en *Machine Leraning* es un tipo de aprendizaje supervisado que se usa para predecir un valor numérico continuo a partir de datos de entrada. A diferencia de la clasificación (que predice categorías), la regresión predice cantidades.

Para entender podemos ver la analogía del proceso para predecir el valor de una casa. Las entradas serían el tamaño, el número de habitaciones, y su ubicación, y la salida sería el precio de la casa en dinero. En este caso, la regresión encuentra la relación matemática entre las entradas y la salida para poder predecir precios de casas nuevas. El proceso general es obtener datos de entrenamiento, con el cual se entrena un modelo que aprende la relación entre entradas y salidas, para finalmente usar el modelo para predecir salidas para nuevos datos de entrada.

En FluCoMa existe una clase llamada `MLPRegressor`, que implementa una Red Neuronal Multicapa (*MLP: Multi-Layer Perceptron*) para hacer regresión. Es decir, aprende a mapear un conjunto de valores de entrada a un conjunto de valores de salida continuos.

```
Capa de entrada → Capas ocultas → Capa de salida

   [x1]                                  [y1]
   [x2]  →  [neurona→][neurona]  →    [y2]
   [x3]                                  [y3]
```

El uso de una `MPL` sirve entre otros para crear *mappings* expresivos entre descriptores de audio (MFCCs), parámetros de síntesis, posición del ratón (x, y), frecuencia, amplitud, timbre, datos de sensores, efectos de audio, análisis espectral, parámetros de filtros y también lo que nos convoca, es decir, gestos y poses del cuerpo.

Sin profundizar en esta etapa cada una de las partes y conceptos de la red neuronal, procederemos a describir el funcionamiento de un `patch` que captura las coordenadas de las poses del cuerpo, cuyos datos son la entrada de la red neuronal para controlar el sonido de un sintetizador. El sonido es mas bien simple pero sirve para mostrar el proceso.

Control de Instrumento Musical con la Pose

Detección de Pose Corporal con MediaPipe

No existe mucha diferencia de la captura de pose con la gestual que hicimos anteriormente con las manos. El código es el siguiente,

```

# importamos las librerías necesarias openCV, MediaPipe y Python-OSC

import cv2 as cv
import time
import mediapipe as mp
from mediapipe.tasks import python
from mediapipe.tasks.python import vision
from pythonosc import udp_client

# Configuración de OSC, IP y puerto. Iniciamos el cliente UPD
OSC_IP = "127.0.0.1"
OSC_PORT = 57120
osc_client = udp_client.SimpleUDPClient(OSC_IP, OSC_PORT)

# definimos en una variable el lugar donde se encuentra el modelo.
model_path = '/home/pemb/python/mediapipe/pose_landmarker_lite.task'

# Iniciamos los Tasks de MediaPipe
BaseOptions = mp.tasks.BaseOptions
PoseLandmarker = mp.tasks.vision.PoseLandmarker
PoseLandmarkerOptions = mp.tasks.vision.PoseLandmarkerOptions
PoseLandmarkerResult = mp.tasks.vision.PoseLandmarkerResult
VisionRunningMode = mp.tasks.vision.RunningMode

# Creamos Una función callback para la instancia con un modo LIVE STREAM
def print_result(result: PoseLandmarkerResult, output_image: mp.Image, timestamp_ms: int):
    #print('pose landmarker result: {}'.format(result))
    if result and result.pose_landmarks:
        # Creamos una lista llamada landmarks-data
        # Creamos una lista que contendrá todos
        # los datos que enviaremos por el mensaje OSC
        landmarks_data = []
        # Ahora procedemos a estructurar nuestros mensaje OSC
        # Para cada landmark encontrado
        for body_idx, landmarkResult in enumerate(result.pose_landmarks):
            # Para cada landmark de cada grupo de landmarks
            for i, land in enumerate(landmarkResult):
                # agregamos las coordenadas x, y, y z a nuestra lista
                landmarks_data.extend([land.x, land.y, land.z])
            osc_client.send_message(f"/landmarks/{body_idx}", landmarks_data)
            # Limpiamos nuestra lista para que se llene
            # con los datos del próximo cuadro
            landmarks_data.clear()

# Declaramos las opciones del detector de poses
options = PoseLandmarkerOptions(
    # Usamos el modelo que se encuentra en el lugar indicado anteriormente
    base_options=BaseOptions(model_asset_path = model_path),
    # Cuantas poses queremos detectar.
    # En este caso una.
    num_poses=1,
    # Como vamos a hacer correr el detector
    # En este caso LIVE_STREAM
    running_mode = VisionRunningMode.LIVE_STREAM,
    # La función callback que usaremos "print result"
    result_callback = print_result)

# Inicializamos la captura de vídeo
cap = cv.VideoCapture(0)
if not cap.isOpened():
    print("Cannot open camera")
    exit()

# Creamos una instancia de PoseLandmaker llamado landmaker
```

```
# usando las opciones anteriores
with PoseLandmarker.create_from_options(options) as landmarker:
 # Creamos un bucle infinito
    while True:
        # captura cuadro por cuadro
        ret, frame = cap.read()
        # Si el cuadro no es recibido correctamente,
        # es decir ret no es true,
        # enviamos un mensaje "error" y salimos del bucle.
        if not ret:
            print("Can't receive frame (stream end?). Exiting ...")
            break

        # Si ret es true, es decir recibimos un cuadro
        # convertimos la imagen del cuadro
        # a un tipo de imagen mediaPipe SRGB
        mp_image = mp.Image(image_format = mp.ImageFormat.SRGB, data = frame)

        # Ingresamos la imagen como parámetro
        # al método recognize_async
        landmarker.detect_async(mp_image, time.time_ns() // 1_000_000)

        # mostramos la imagen de la cámara
        cv.imshow('frame', frame)

        # si pulsamos la tecla "q" salimos del bucle
        if cv.waitKey(1) == ord('q'):
            break
# Cerramos la captura
# y destruimos todas las ventanas abiertas
cap.release()
cv.destroyAllWindows()
```

Regresión usando SuperCollider y FluCoMa

El programa que desarrollaremos en SuperCollider recibirá los datos de los `landmarks` encontrados por MediaPipe. El primer paso será filtrar los datos de la componente `z` de las coordenadas de los `landmarks` para disminuir la cantidad de información. Esto no es necesario, pero para ser usado como controlador de sonido, con las componentes `x` e `y` de cada `landmark` es suficiente. En total MediaPipe nos entrega 33 `landmarks`. Cada uno de estos `landmarks` tienen sus coordenadas `x`, `y`, y `z`, por lo que recibimos desde el programa de Python de MediaPipe 99 datos. Si filtramos la componente `z` de cada *landmark* terminaremos con 66 datos de entrada para nuestra red neuronal. Por otro lado, buscamos controlar un instrumento que tienen 11 parámetros de entrada, por lo que nuestra salida de la red neuronal sera de 11 dimensiones.

El programa usa variadas clases para crear una interfaz gráfica que nos permita fácilmente navegar por el proceso de visualización, entrenamiento e inferencia. El resultado final visualmente sería la siguiente,

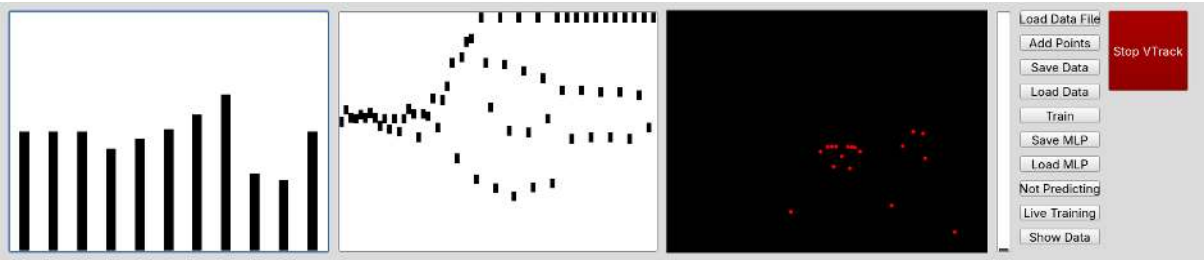


Figura 4: Imagen del programa de regresión de aprendizaje automático usando Supercollider

A continuación procederemos a describir el programa en su totalidad. Este ejemplo nos servirá no solo como implementamos una regresión sino que además nos mostrará mucho de lo que hablamos al comienzo pasando información desde el lenguaje a el servidor de sonido y vice versa.

El código esta todo en un bloque, de tal forma que a diferencia de la implementación de la navegación del corpus de data de sonidos, se ejecutará todo y no tendremos que por parte por parte. Esto además significa que no usamos variables globales, solo locales.

```
(
// Definimos una variable "synth" para nuestro instrumento, ima "uv" para un UserView,
var synth, uv, mss, msmp, msdel, sdelta, win, xyslider, mpdict;

// Creamos dos FluidDataSet. Uno para los datos
// provenientes de MediaPipe a través de mensajes OSC
// y otro para los datos de salida de la red neuronal
var mpdata = FluidDataSet(s);
var outdata = FluidDataSet(s);

// Variable que contiene el número de elementos de los Array
var numValues = 66;

// Creamos dos Array y los inicializamos en 0
// para los datos que provienen de MediaPipe
// El array tiene 66 elementos
var adata = Array.fill(numValues, 0);

// Creamos dos Buffer para los datos de salida con 11 elementos
// y los datos de entrada de 66 elementos de la red neuronal

var mpbuf = Buffer.alloc(s, 66);
var paramsbuf = Buffer.alloc(s,11);

// creamos una variable llamada counter,
// una variable llamada predicting
// y otra booleana llamada predict,
var counter = 0;
var predicting = 0;
var predict = false;

// Creamos uns variable booleana llamada visibility;
var visibility = true;

// Creamos una función circle
// cuyos argumentos son las coordenadas x e y
// y el radio del circulo

var circle = {
  |x, y, r|
  Pen.fillColor = Color.new255(255, 0, 0, 255);
  Pen.strokeColor = Color.red;
  Pen.addArc(x@y + ((r@r)), r, 0, 2pi);
  Pen.fillStroke;
};

// La declaración del Fluid Regressor.

// (1) el primer argumento s, es el servidor de audio de SC. El modelo vive y opera dentro del
      servidor.

// (2) El segundo argumento [7] es la Arquitectura de las capas ocultas de la red neuronal.
     [7] significa 1 capa con 7 neuronas, [7, 7] serían 2 capas de 7 neuronas cada una, [14, 7,
      4] serían 3 capas ocultas con 14, 7 y 4 neuronas respectivamente. Más neuronas y capas =
     modelo más complejo, pero también más lento de entrenar y con más riesgo de sobre ajuste (
     overfitting). En general, estructuras como [7] es simple y buena para mappings sencillos;
     estructuras parecidas a [7, 7] tienen más capacidad de aprendizaje; estructuras como [14,
     7] en donde la primera capa es más ancha y luego se va reduciendo sirve para reducir la
     dimensionalidad, y finalmente estructuras como [4, 8, 4] se les llaman forma de "diamante
     ".
```

```
52
53 //(3) El tercer argumento activation es la función de activación de las capas ocultas. Define
     cómo cada neurona transforma su entrada antes de pasarla a la siguiente capa. sigmoid
     transforma cualquier número a un valor entre 0 y 1. En general FluidMLPRegressor.sigmoid
     tienen un rango de entre 0 a 1 y sirve para mappings suaves y graduales. FluidMLPRegressor
     .relu que tienen un rango de entre 0 a ∞, sirve para redes más profundas, y es más rápido.
      FluidMLPRegressor.tanh que tienen un rango de entre -1 a 1 sirve cuando los datos tienen
     valores negativos, y FluidMLPRegressor.identity que tienen un rango de entre ∞- a ∞ no
     produce transformación.
54
55 // (4) El cuarto argumento el outputActivation es la función de activación de la capa de
     salida. Determina el rango de los valores que el modelo puede predecir. Usando sigmoid
     aquí significa que todas las salidas estarán entre 0 y 1, lo cual es muy útil si tus datos
      de salida están normalizados en ese rango. Las otras alternativas de rango corresponden a
      las posibilidades de la activación de las capas escondidas: Si la salida es entre 0 y 1
     el outputActivation debe ser FluidMLPRegressor.sigmoid. Si la salida puede ser cualquier
     valor, el outputActivation puede ser FluidMLPRegressor.identity, si la salida es entre -1
     y 1, el outputActivation debe ser FluidMLPRegressor.tanh.
56
57 // (5) El quinto argumento corresponde a maxIter: 1000, que refiere al número máximo de
     iteraciones de entrenamiento (también llamadas épocas). El modelo ajusta sus pesos
     internos en cada iteración.(5) El sexto argumento, learnRate, es la tasa de aprendizaje,
     controla qué tan grandes son los ajustes que hace el modelo en cada iteración. Con un
     valor muy alto, el modelo aprende rápido pero puede ser inestable y no converger, con un
     valor muy bajo, el modelo aprende lento pero de forma más estable. 0.1 es un valor
     relativamente alto, bueno para datasets pequeños.
58
59 // (6) El sexto argumento batchSize, define cuántos ejemplos se usan para calcular cada
     actualización de los pesos de la red. Un valor 1 SGD, bueno para datasets pequeños, un
     valor 8 es un mini-batch pequeño, y un valor 32, es un mini-batch común en deep learning.
     En FluCoMa con datasets pequeños (como en performances en vivo), batchSize: 1 es muy común
      y funciona bien.
60
61 // (7) El septimo argumento, validation, es la fracción de datos que se reserva para
     validación durante el entrenamiento. Una validación 0 significa que no hay validación, usa
      todos los datos, una 0.1 reserva 10% para validación, una 0.2 reserva 20% para validación
     . La validación sirve para detectar sobreajuste (overfitting), es decir, cuando el modelo
     aprende los datos de entrenamiento de memoria pero no generaliza bien. Con validation: 0
     el modelo usa todos los datos disponibles para aprender, lo cual es útil cuando tienes
     pocos datos de entrenamiento.
62
63
64 var mlp = FluidMLPRegressor(
65          s,
66   [7],
67   activation:FluidMLPRegressor.sigmoid,
68   outputActivation:FluidMLPRegressor.sigmoid,
69   maxIter: 1000,
70   learnRate:0.1,
71   batchSize:1,
72   validation:0
73 );
74
75 // Definimos la función OSCdef en donde recibimos los datos de MediaPipe
76
77 OSCdef(\oscposes, {
78   arg msg, mmsg;
79
80         // variable que guarda un array desde los mensaje OSC descontando el primer elemento,
     que es la dirección OSC del mensaje.
81
82         var incarray = msg[1..];
83
84         // Limpiamos el Array cada vez antes de copiar los datos proveniente de MediaPipe para
      no agregarlos a los ya existente.
85   adata.clear;
86
87   // Filtramos la coordenada Z, con el método reject, cuyo parámetro es una función. En esta
     definimos una operación que elimina cada tercera componente del array, es decir la
     coordenada Z. esto lo copiamos al array adata
88
89   adata = incarray.reject({
90     arg item, i;
```

```
    (i + 1) % 3 == 0 }
  );

// Copiamos el contenido del array adata al Buffer mpbuf desde el índice 0 del Buffer.
  mpbuf.setn(0, adata);

// Preguntamos si la variable predict que corresponde a si queremos entrenar el modelo o esta en modo inferencia es verdad. Si esta es verdad estamos en modo inferencia.
  if(predict == true, {
  // si estamos en modo inferencia paramos los datos del array a un multislider que alojamos en la variable msmp. Este proceso lo hacemos en una función con el metido defer debido a que es un proceso gráfico.
                {msmp.valueAction_(adata)}.defer;
// Finalmente actualizamos el UserView en donde dibujaremos multiples circulos que corresponden a las articulaciones de nuestra pose.
    defer{ uv.refresh };
  });
}, '/landmarks/0');

// Definición de la Interfaz Gráfica

//Create window

//Cerramos cualquier ventana de SC abierta
Window.closeAll;

// Asignamos una ventana de dimensiones de 1500 píxeles de ancho por 320 píxeles de alto
win = Window("Training Window", Rect(10, 10, 1500, 320))
.front
.alwaysOnTop_(true);

// Creamos un MultiSliderView de 400 x 300 píxeles y se lo asignamos a la variables mss
// En su argumento action, que requiere una función, recibimos los valores del multiSliderView y se los asignamos al parambuf, que es el Buffer en donde alojemos las variables para el instrumento. Esto lo repetimos por 11 para coincidir con el tamaño del Buffer.

mss = MultiSliderView(win, Rect(10, 10, 400, 300))
.elasticMode_(1)
.isFilled_(1)
.action_({
  arg ms;
  paramsbuf.setn(0, ms.value);
}).value_(0.5.dup(11));

// Creamos otra instancia de MultiSliderView y se lo asignamos a la variable msmp. Le asignamos sus valores con los valores del Array adata, es decir, con los valores que vienen de MediaPipe.

msmp = MultiSliderView(win, Rect(420, 10, 400, 300));
msmp.value_(adata);
msmp.elasticMode_(1);
msmp.isFilled_(0);
msmp.drawLines_(0);
msmp.drawRects_(1);
msmp.showIndex = false;
msmp.doAction;

// Creamos una instancia de UserView y la asignamos a la variable uv. En su función .drawFunc usamos cada par del array adata para usarlo como las coordenada de los círculos. Como estas están normalizadas las multiplicamos por el ancho y alto del UserView, solo si la variable booleana visibility es True.

uv = UserView(win, Rect(830, 10, 400, 300));
uv.background_(Color.black);
uv.drawFunc = { |uvview|
  if(visibility == true){
    adata.pairsDo{ |a, b|
      circle.value(
        a * uv.bounds.width,
        b * uv.bounds.height,
        2
      );
          };
```

```
  };
};

// Declaramos un botón cuya acción es
// cargar un punto en el FluidDataSet "mpdata"
// con los datos del Buffer "mpbuf', que contiene los
// datos del Array "adata"
// Por otro lado la acción carga en el FluidDataSet
// "outdata" un punto con los valores contenidos en el
// Buffer "paramsbuf" que ha recibido los datos del
// MultiSliderView "mss".
// Cada FluidDataSet comparte el mismo identificador
// construido en la variable "id". De esta forma el
// FluidMLPRegressor reconoce que los datos de los
// diferentes datasets están asociados.

Button(win,Rect(1270,40,100,20))
.states_([["Add Points"]])
.action_{
  var id = "point-%".format(counter);
  mpdata.addPoint(id, mpbuf);
  outdata.addPoint(id, paramsbuf);
  counter = counter + 1;
  mpdata.print;
  outdata.print;
};

// Declaramos botones para guardar y cargar datos
// en y desde archivo JSON

// Declaramos otras instancia de la clase Button
// cuya acción es guardan en dos archivo JSON
// separados los dos FluidDataSet a los cuales
// les hemos agregado los datos

Button(win,Rect(1270,70,100,20))
.states_([["Save Data"]])
.action_{
  FileDialog({
    arg folder;
    folder.postln;
    mpdata.write(folder+/+"xydata.json");
    outdata.write(folder+/+"paramsdata.json");
  }, {}, 2, 0, true);
  };

// Declaramos otras instancia de la clase Button
// cuya acción es cargar los archivos JSON
// de los dos FluidDataSet

// Estos datos son cargados en los FluidDataSet
// "mpdata" y "outdata" respectivamente


Button(win,Rect(1270,100,100,20))
.states_([["Load Data"]])
.action_{
  FileDialog({
    arg folder;
    mpdata.read(folder+/+"xydata.json");
    outdata.read(folder+/+"paramsdata.json");
  }, fileMode: 2, acceptMode: 0, stripResult: true);
};

// Declaramos otras instancia de la clase Button
// cuya acción es llamar al método fit de la
// instancia FluidMLPRegressor "mlp"
// Los argumentos de este método son los
// FluidDataSet "mpdata" y "outdata"
// La salida de este método la posteamos
// en la consola y significa el error.
// A menor el número de "loss", mas ha
// aprendido la red neuronal. Sin embargo, no queremos
```

```
// llegar a 0, para no producir un overfitting.
// El overfitting (o sobreajuste) ocurre cuando un modelo de machine learning
// aprende los datos de entrenamiento demasiado bien,
// al punto de memorizar hasta el ruido y las irregularidades,
// perdiendo la capacidad de generalizar con datos nuevos.

Button(win,Rect(1270,130,100,20))
.states_([["Train"]])
.action_{
  mlp.fit(mpdata, outdata,{
    arg loss;
    loss.postln;
  });
};

// Declaramos otras instancia de la clase Button
// cuya acción es guardar un archivo JSON que
// contiene el modelo entrenado.

Button(win,Rect(1270, 160, 100, 20))
.states_([["Save MLP"]])
.action_{
  Dialog.savePanel({
    arg path;
    if(PathName(path).extension != "json"){
      path = "%.json".format(path);
    };
    mlp.write(path);
  });
};

// Declaramos otras instancia de la clase Button
// cuya acción es cargar un archivo JSON que
// contiene un modelo pre-entrenado.

Button(win,Rect(1270, 190, 100, 20))
.states_([["Load MLP"]])
.action_{
  Dialog.openPanel({
    arg path;
    mlp.read(path);
  });
};

// Declaramos otras instancia de la clase Button
// cuya acción cambiar el modo, desde no predecir,
// es decir entrenar, a predecir, es decir inferir.

// Con esta información no solo cambiamos la variable
// boleana "predict", sino que también enviamos el valor
// de la variable "predicting" al instrumento.

Button(win,Rect(1270, 220, 100, 20))
.states_([["Not Predicting"],["Predicting"]])
.action_{
  arg but;
  if(but.value == 1){predict = true}{predict == false};
  synth.set(\predicting, but.value);
};

// Declaramos otras instancia de la clase Button
// cuya acción es decirle que dibuje o no los
// puntos de las articulaciones entregadas por MediaPipe

Button(win,Rect(1270,280,100,20))
.states_([["Show Data"],["Hide Data"]])
.action_{
  arg msg;
  visibility = msg.value.asBoolean;
  visibility.postln;
};

// Finalmente, declaramos otras instancia de la clase Button
```

```
// cuya acción es hacer correr y parar el python script de MediaPipe
// desde la interfaz gráfica de Supercollider

Button(win,Rect(1380,10,100,100))
.states_([
        ["Start VTrack", Color.white, Color.green(0.6)],
        ["Stop VTrack",  Color.white, Color.red(0.6)],
    ])
.action_{
  arg msg;
  if (msg.value == 1) {
    ~pid = "bash -c 'conda run -n mediapipe python /camino/a/bodyPose.py'".unixCmd;
        } {
            "pkill -f bodyPose.py".unixCmd;  // kill it on toggle off
        };
    });

};

// Definimos el instrumento y lo guardamos
// en la variable synth

synth = {

        // Definimos el argumento predicting.
  arg predicting = 0;

  var in, latent, modLatent, prior, rlmod = 20;
  var osc1, osc2, feed1, feed2, base1=69, base2=69, base3 = 130, loud1, loud2, sig;
  var val, mp, trig;

  // Convertimos "mpbuf" a señales de control
  mp = FluidBufToKr.kr(mpbuf);
  // Disparamos un trigger si existe un cambio en los mensajes de control
  trig = Mix(Changed.kr(mp));
  // El trigger es activado sólo cuando el argumento
  // "predicting" es 1. De otra forma el trigger será
  // 0, por lo tanto no activando nada.
  // La instancia "mlp" The mlp calcula la salida de la red neuronal
  // únicamente cuando es activada hacemos la inferencia.
        // Los argumentos son el trigger, el Buffer de entrada y el Buffer de salida
  mlp.kr(trig * predicting, mpbuf, paramsbuf);
        // Usando la instancia de clase FluidBufToKr convertimos los valores del Buffer
        // como un Array de valores de control
  val = FluidBufToKr.kr(paramsbuf);
        // Definimos un mensaje OSC para enviar los valores
        // inferidos que están guardados en parambuffer con
        // la ruta /paramsChanged, cada vez que los valores
        // del buffer parambuf cambian.
        // Esto es necesario porque los valores están en el servidor de sonido
        // y necesitamos enviarlos a nivel de lenguaje como array
        // para poder modificar el MultiFaderView de los parámetros.
  SendReply.kr(Mix(Changed.kr(val)), "/paramsChanged", val);

        // Acá comienza los UGen generadores de sonido

  #feed2, feed1 = LocalIn.ar(2);

  // Descomentamos para ver si estamos recibiendo datos de la red neuronal
  // FluidBufToKr.kr(~mpbuf).poll;

  loud1 = FluidLoudness.kr(feed1,[\loudness],1,0,hopSize:64);
  loud2 = FluidLoudness.kr(feed2,[\loudness],1,0,hopSize:64);

        // Fijense como accedemos a los valores del array "val" con los índice
  osc1 = SinOsc.ar(
            (((feed1 * val[0]) +  val[1]) * base1).midicps,
            mul:(val[2] * 50).dbamp).atan;
        osc1 = MoogFF.ar(
            osc1,
            (base3 - (val[3]*(loud2.clip(-120,0)+120))).lag(128/44100).midicps,
            val[4]*3.5);
  osc2 = SinOsc.ar(
```

```
            (((feed2 * val[5]) +  val[6]) * base2).midicps,
            mul: (val[7] * 50).dbamp).atan;
  osc2 = MoogFF.ar(
            osc2,
            (base3 - (val[8] * (loud1.clip( -120, 0) + 120))).lag(128 / 44100).midicps,
            val[9] * 3.5);
  LocalOut.ar([osc1,osc2]);
sig = [osc1, osc2] * 0.1;
}.play;

// Finalmente definimos una función OSCdef
// para recibir los mensajes del servidor de sonido
// al interprete del lenguaje para asignarlo al
// MuiltiSliderView "mss"
OSCdef(\paramsChanged,{
  arg msg;
  defer{
    mss.value_(msg[3..]);
  };
},"/paramsChanged");
)
```

Conclusion: Posibles expansiones

En esta tutoría hicimos un repaso rápido pero conciso de la instalación e implementación de sistemas de control gestual y machine learning para navegar un archivo y controlar la generación de sonidos. Se espera que desde los ejercicios hechos se abran muchas posibilidades. Algunos de las implementaciones que se pueden hacer con las herramientas entregadas es por ejemplo entrenar nuestros propios gestos usando una red neuronal para después controlar, con la salida de esta red, los puntos de navegación del corpus de datos. Otra posibilidad es generar una reducción dimensional a 3 dimensiones para producir navegaciones tridimensionales con la ayuda de otras herramientas como openFrameworks. A su vez, se pueden expandir los ejercicios. Por ejemplo podemos agregar en el proceso de segmentación de los archivos de audio el nombre del archivo de tal forma que podamos acceder a este desde la interfaz, o entrenar una red neuronal para clasificar eventos sonoros específicos, en donde no solo son ordenados y agrupados para su navegación, sino que además identificados. Por último, existe la posibilidad de probar con los diferentes parámetros de las clases de MediaPipe, FluCoMa y Supercollider. De esta forma, se abren un sinnúmero de posibilidades para expandir el uso de estas herramientas.